



Developing Productions

Version 2026.1
2026-04-20

Developing Productions

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Introduction to Developing Productions	1
1.1 Environmental Considerations	1
1.1.1 Production-enabled Namespaces	1
1.1.2 Web Application Requirement	1
1.1.3 Reserved Package Names	1
1.2 A Look at a Production Definition	2
1.3 Development Tools and Tasks	3
1.3.1 Portal Tasks	3
1.3.2 IDE Tasks	3
1.4 Available Specialized Classes	4
2 Programming Business Services, Processes, and Operations	5
2.1 Introduction to Programming Business Hosts	5
2.2 Key Principles for Productions	6
2.3 Passing Values by Reference or as Output	7
2.3.1 Typical Callback Method	7
2.3.2 Typical Helper Method	8
2.4 Accessing Properties and Methods from a Business Host	8
2.5 Accessing Production Settings	9
2.6 Choosing How to Send Messages	9
2.6.1 Synchronous and Asynchronous Sending	9
2.6.2 Deferred Sending	10
2.7 Generating Alerts	11
2.8 See Also	12
3 Adding Logging and Tracing	13
3.1 Generating Event Log Entries	13
3.1.1 Generating Event Log Entries in ObjectScript	13
3.2 Adding Trace Elements	14
3.2.1 Writing Trace Messages in ObjectScript	14
3.2.2 Writing Trace Messages from BPL or DTL	15
3.3 See Also	15
4 Adding and Removing Settings	17
4.1 Adding Settings	17
4.2 Removing Settings	17
4.3 Specifying Categories and Controls	18
4.4 Available Categories	18
4.5 Available Controls	19
4.5.1 The selector Control	19
4.5.2 Examples for Settings Controls	20
4.6 Specifying Default Values	21
4.7 Considerations When Redefining Settings	22
5 Defining Messages	23
5.1 Types of Message Bodies	23
5.2 Creating a Simple Message Body Class	23
5.3 Message Storage	24
5.4 Controlling the Display	25

5.5 Creating a Complex Message Body Class	25
5.6 Specifying Message Purge Behavior	26
5.7 See Also	27
6 Defining Business Services	29
6.1 Introduction to Defining Business Services	29
6.2 Key Principles for Business Services	30
6.3 Defining a Business Service Class	30
6.4 Implementing the OnProcessInput() Method	31
6.5 Sending Request Messages	31
6.5.1 Using the SendRequestSync() Method	32
6.5.2 Using the SendRequestAsync() Method	32
6.5.3 Using the SendDeferredResponse() Method	32
6.6 Sending Multiple Request Messages	33
6.7 Processing Only One Event Per Call Interval	33
7 Defining Business Processes	35
7.1 Introduction to Defining Business Processes	35
7.2 Comparison of Business Logic Tools	36
7.3 Key Principles for Business Processes	37
7.4 Defining BPL Business Processes	38
7.5 Defining Custom Business Processes	38
7.5.1 Implementing the OnRequest() Method	38
7.5.2 Implementing the OnResponse() Method	39
7.5.3 Methods to Use in OnRequest() and OnResponse()	40
8 Defining Business Operations	41
8.1 Introduction to Defining Business Operations	41
8.2 Key Principles for Business Operations	42
8.3 Defining a Business Operation Class	42
8.4 Defining a Message Map	43
8.5 Defining Message Handler Methods	44
8.6 Business Operation Properties	45
8.7 Calling Adapter Methods	45
8.8 Sending Requests to Targets within the Production	46
8.8.1 The DeferResponse() Method	46
8.9 Suspending Messages	46
9 Defining an Alert Processor	49
9.1 Background Information	49
9.2 Using a Simple Email Alert Processor	49
9.3 Using a Simple Outbound Adapter Alert Processor	50
9.4 Using a Routing Alert Processor	50
9.4.1 Defining the Alert Processor as a Routing Process	50
9.4.2 Defining the Business Operations	51
9.5 Adding Custom Code to Alert Management	51
9.5.1 Alert Manager	51
9.5.2 Notification Manager	52
9.5.3 Alert Monitor	52
9.5.4 Notification Operation	53
10 Defining Data Transformations	55
10.1 Introduction to Defining Data Transformations	55

10.2 Defining DTL Transformations	55
10.3 Defining Custom Transformations	55
11 Defining Business Metrics	57
11.1 Introduction to InterSystems IRIS Business Metrics	57
11.1.1 Business Metric Properties	57
11.1.2 Single- and Multi-instance Business Metrics	58
11.1.3 Business Metrics as Business Services	60
11.2 Defining a Single-instance Business Metric	60
11.2.1 Defining Simple Business Metric Properties	60
11.2.2 Defining Business Metric Properties with Autohistory	61
11.2.3 Specifying Other Parameters for Metric Properties	61
11.2.4 Assigning Values to Business Metric Properties	62
11.3 Defining a Multi-instance Business Metric	63
11.3.1 Defining a Static Set of Instance Names	63
11.3.2 Defining the Instance Names Dynamically	64
11.3.3 Assigning Values to Properties in a Multi-instance Metric	64
11.4 Other Options in Business Metrics	65
11.4.1 Defining Actions for Use in Dashboards	65
11.4.2 Implementing OnInit()	65
11.5 Adding Business Metrics to Dashboards	65
11.6 Adding Business Metrics to the Production Monitor	65
11.7 Setting and Getting Values Programmatically	66
11.7.1 Using the GetMetric() Method	66
11.7.2 Using the SetMetric() Method	67
11.8 About the Business Metric Cache	67
12 Defining an Enterprise Message Bank	69
12.1 Enterprise Message Bank Overview	69
12.2 Defining the Message Bank Server	70
12.3 Adding a Message Bank Helper Class	71
12.4 Notes about the Message Bank	72
13 Less Common Tasks	73
13.1 Defining Custom Utility Functions	73
13.2 Rendering Connections When the Targets Are Dynamic	75
13.3 Using Ens.Director to Start and Stop a Production	76
13.4 Using Ens.Director to Access Settings	77
13.5 Invoking a Business Service Directly	79
13.6 Creating or Subclassing Inbound Adapters	80
13.6.1 Introduction to Inbound Adapters	80
13.6.2 Defining an Inbound Adapter	80
13.6.3 Implementing the OnTask() Method	81
13.7 Creating or Subclassing Outbound Adapters	82
13.7.1 Introduction to Outbound Adapters	82
13.7.2 Defining an Outbound Adapter	82
13.8 Including Credentials in an Adapter Class	83
13.9 Overriding Production Credentials	83
13.10 Overriding Start and Stop Behavior	84
13.10.1 Callbacks in the Production Class	84
13.10.2 Callbacks in Business Host Classes	84
13.10.3 Callbacks in Adapter Classes	84

13.11 Programmatically Working with Lookup Tables	85
13.12 Defining a Custom Archive Manager	86
14 Testing and Debugging Productions	87
14.1 Correcting Production Problem States	87
14.1.1 Suspended Productions	87
14.1.2 Recovering a Troubled Production	87
14.1.3 Resetting Productions in a Namespace	87
14.2 Testing from the Management Portal	88
14.2.1 Using the Testing Service	88
14.3 Debugging Production Code	90
14.4 Enabling %ETN Logging	90
15 Deploying a Production	91
15.1 Overview of Deploying a Production	91
15.2 Exporting a Production	92
15.3 Deploying a Production on a Target System	95
Appendix A: Enabling Tracing Code	97
A.1 Overview	97
A.2 Enabling Tracing	97
A.3 Enabling Logging for Trace Messages	99
A.4 Seeing Trace Messages in the Terminal	99
Appendix B: Life Cycle of a Production and Its Parts	101
B.1 Life Cycle of a Production	101
B.1.1 Production Startup	101
B.1.2 Production Shutdown	101
B.2 Life Cycle of a Business Service and Adapter	102
B.2.1 Production Startup	102
B.2.2 Runtime	103
B.2.3 Production Shutdown	103
B.3 Life Cycle of a Business Process	104
B.3.1 Life Cycle in the Public Queue	104
B.3.2 Life Cycle in a Private Queue	105
B.4 Life Cycle of a Business Operation and Adapter	105
B.4.1 Production Startup	105
B.4.2 Runtime	106
B.4.3 Production Shutdown	106

1

Introduction to Developing Productions

This page introduces the process of developing [interoperability productions](#).

For information on how productions work, see [Core Concepts When Monitoring Interoperability Productions](#).

1.1 Environmental Considerations

You can use productions only within an [interoperability-enabled](#) namespace that has a [specific](#) web application. When you create classes, you should avoid using [reserved package names](#). The following subsections give the details.

1.1.1 Production-enabled Namespaces

An *interoperability-enabled namespace* is a namespace that has global mappings, routine mappings, and package mappings that make the classes, data, and menus that support productions available to it. For general information on mappings, see [Configuring Namespaces](#). (You can use the information in that section to see the actual mappings in any interoperability-enabled namespace; the details may vary from release to release, but no work is necessary on your part.)

The system-provided namespaces that are created when you install InterSystems IRIS are not interoperability-enabled, except, on the community edition, the USER namespace is an interoperability-enabled namespace. Any new namespace that you create is by default interoperability-enabled. If you clear the **Enable namespace for interoperability productions** check box when creating a namespace, InterSystems IRIS® data platform creates the namespace with productions disabled.

Important: All system-provided namespaces are overwritten upon reinstallation or upgrade. For this reason, InterSystems recommends that customers always work in a new namespace that you create. For information on creating a new namespace, see [Configuring Data](#).

1.1.2 Web Application Requirement

Also, you can use a production in a namespace only if that namespace has an associated web application that is named `/csp/namespace`, where namespace is the namespace name. (This is the default web application name for a namespace.) For information on defining web applications, see [Applications](#).

1.1.3 Reserved Package Names

In any [interoperability-enabled namespace](#), avoid using the following package names: Demo, Ens, EnsLib, EnsPortal, or CSPX. These packages are completely replaced during the upgrade process. If you define classes in these packages, you would need to export the classes before upgrading and then import them after upgrading.

Also, InterSystems recommends that you avoid using any package names that *start* with Ens (case-sensitive). There are two reasons for this recommendation:

- When you compile classes in packages with names that start with Ens, the compiler writes the generated routines into the ENSLIB system database. (The compiler does this because all routines with names that start with Ens are mapped to that database.) This means that when you upgrade the instance, thus replacing the ENSLIB database, the upgrade removes the generated routines, leaving only the class definitions. At this point, in order to use the classes, it is necessary to recompile them.

In contrast, when you upgrade the instance, it is not necessary to recompile classes in packages with names that do not start with Ens.

- If you define classes in packages with names that start with Ens, they are available in all interoperability-enabled namespaces, which may or may not be desirable. One consequence is that it is not possible to have two classes with the same name and different contents in different interoperability-enabled namespaces, if the package name starts with Ens.

1.2 A Look at a Production Definition

Although you create and configure productions in the Management Portal, it is instructive to get started by looking at the definition of a existing production class in your choice of IDE. This following shows a simple example of a production:

Class Definition

```
Class Demo.FloodMonitor.Production Extends Ens.Production
{
XData ProductionDefinition
{
<Production Name="Demo.FloodMonitor.Production">
  <ActorPoolSize>1</ActorPoolSize>
  <Item Name="Demo.FloodMonitor.BusinessService"
    ClassName="Demo.FloodMonitor.BusinessService"
    PoolSize="1" Enabled="true" Foreground="false" InactivityTimeout="0">
  </Item>
  <Item Name="Demo.FloodMonitor.CustomBusinessProcess"
    ClassName="Demo.FloodMonitor.CustomBusinessProcess"
    PoolSize="1" Enabled="true" Foreground="false" InactivityTimeout="0">
  </Item>
  <Item Name="Demo.FloodMonitor.GeneratedBusinessProcess"
    ClassName="Demo.FloodMonitor.GeneratedBusinessProcess"
    PoolSize="1" Enabled="true" Foreground="false" InactivityTimeout="0">
  </Item>
  <Item Name="Demo.FloodMonitor.BusinessOperation"
    ClassName="Demo.FloodMonitor.BusinessOperation"
    PoolSize="1" Enabled="true" Foreground="false" InactivityTimeout="0">
  </Item>
</Production>
}
```

Note the following points:

- The production is a class, specifically is a subclass of Ens.Production.
- The XData ProductionDefinition block holds the configuration information for the production.
- Each <Item> is a business host; these are also called *configuration items*.
- Each business host refers to a class. ClassName specifies the class on which this host is based. This means that when the production creates an instance of this business host, it must create an instance of the specified class.

- The Name of a business host is an arbitrary string. Sometimes, it can be convenient to use the class name for this purpose, as in this example. This convention does not work when you create a large number of business hosts that use the same class.

It is important to establish naming conventions at an early point during development. See [Best Practices for Creating Productions](#). An absence of naming conventions will lead to confusion.

- The other values in the XData block are all settings. At the top, <ActorPoolSize> is a setting for the production. Within the business host definitions, PoolSize, Enabled, Foreground, and InactivityTimeout are settings for those business hosts.

1.3 Development Tools and Tasks

Productions consist primarily of class definitions and some supporting entities. The process of creating a production can require a small amount of programming or possibly a large amount, depending on your needs. As noted earlier, InterSystems IRIS provides graphical tools that enable nontechnical users to create business logic visually. These tools generate class definitions as needed.

While you develop a production, you use both the [Management Portal](#) and your choice of [IDE](#) as follows.

1.3.1 Portal Tasks

In the Management Portal, you use UIs to define and compile the following:

- [Productions](#)
- [BPL business processes](#)
- [DTL transformations](#)
- [Business rules](#)

You also use the Management Portal for the following additional tasks:

- Defining reusable items for use in settings; these include production credentials, business partners, and so on. See [Configuring Productions](#).
- Starting and stopping the production. See [Starting and Stopping Productions](#).
- Examining the message flow, as part of your testing and debugging process. See [Monitoring Productions](#).
- Testing specific business hosts. See [Testing and Debugging](#).

1.3.2 IDE Tasks

In an IDE, you define and compile the following classes:

- [Message](#) classes.
- [Business service](#) classes. Note that InterSystems IRIS provides business service classes that directly use specific adapters. You might be able to use one of them rather than create your own.
- Custom [business Process](#) classes.
- [Business operation](#) classes. Note that InterSystems IRIS provides business operation classes that directly use specific adapters. You might be able to use one of them rather than create your own.

- Custom [data transformation](#) classes.
- Custom adapter classes. See [Less Common Tasks](#).

Also see other topics in [Less Common Tasks](#).

1.4 Available Specialized Classes

InterSystems IRIS provides many specialized adapter and business host classes that can reduce your development and testing time. For a summary of the most common options, see [Other Production Options](#).

2

Programming Business Services, Processes, and Operations

This page discusses common programming tasks and topics when developing business services, processes, and operations for productions.

2.1 Introduction to Programming Business Hosts

When you create business host classes or adapter classes, you typically implement callback methods, define additional methods as needed, and add or remove settings.

Within a business host or adapter, your methods typically do some or all of the following tasks:

- Get or set values of properties of the business host or adapter.
- Define callback methods. A *callback method* is an inherited method that does nothing by default; your task would be to override and implement this method.
- Execute inherited helper methods of the business host or adapter. *Helper method* is an informal term for a method meant for use by other methods.

In business operations and business processes, your methods typically invoke inherited methods to send messages to other business hosts within the production.

- Generate log, alert, or trace notifications in response to various conditions that may arise. InterSystems IRIS uses the following terminology:
 - *Log entries* are intended to note items such as external, physical problems (for example, a bad network connection). InterSystems IRIS® data platform automatically writes these to the Event Log.
 - *Alerts* are intended to alert users, via an *alert processor* that you define and add to the production. InterSystems IRIS also automatically writes these to the Event Log.
 - *Trace items* are meant for debugging and diagnostic purposes. You can use these, for example, to locate program errors before deploying the production. InterSystems IRIS can write these to the Event Log, to the Terminal, or both.

Not all types of error or activity should necessarily generate these notifications. It is up to the developer to choose which occurrences to record, and how to record them. Note that the Event Log should not register program errors; these should be resolved before the production is released.

2.2 Key Principles for Productions

It is important to understand the programming practices that are best suited within productions. Business hosts execute in separate processes, which means that you should make sure that:

- If a business host starts a transaction, the same business host should complete it or roll it back.
To be more specific, if your business host code starts a transaction, such as by an ObjectScript `TSTART` or a SQL statement with `%COMMITMODE = EXPLICIT`, the same business host requires code to complete or roll back that transaction. When nesting transactions, be aware that the process will hold all locks until all parts of the transaction are either committed or rolled back.
- If a business host allocates any system resources (such as taking out locks or opening devices), the same business host should release them.
- Any information that is to be shared between business hosts should be carried within a message sent between them (rather than via public variables).

Important: Failure to follow these guidelines can cause your production to become inoperable.

Similar considerations apply to business rules and data transformations.

Also, you must often handle error codes received from InterSystems IRIS methods. The InterSystems IRIS Interoperability development framework is designed to allow your custom code to be simple and linear as possible with regard to error codes. For example:

- The **OnProcessInput()** method of business services and the **OnMessage()** and other user-written MessageMap methods of business operations are wrapped by the production framework, so that you do not need to include any additional error trapping in your code.
- The adapter methods available for use by custom business operation code are guaranteed never to trap out, as are the framework methods available to custom code, such as **SendAlert()** and **SendRequestSync()**.
- The adapter methods automatically set suitable values for the Retry property of the business host when error conditions do occur.

The details vary by adapter, but the common principle is to cause a retry for temporary errors and for errors that allow manual intervention. For example, if the HTTP outbound adapter times out, it will automatically set Retry equal to 1, for the associated business operation. If this is not suitable for your use case, your custom code can set Retry equal to 0.

Given these precautions built into the production framework, InterSystems recommends that for normal circumstances your custom code should simply check the error code of each call, and if it is an error value, quit with that value. The following is an example of this coding style:

Class Definition

```

Class Test.FTP.FileSaveOperation Extends Ens.BusinessOperation
{
  Parameter ADAPTER = "EnsLib.File.OutboundAdapter";

  Method OnMessage(pRequest As Test.FTP.TransferRequest,
                  Output pResponse As Ens.Response) As %Status
  {
    Set pResponse=$$NULLOREF
    Set tFilename=..Adapter.CreateTimestamp(pRequest.Filename, "%f_%Q")
    ; file with timestamp should not already exist
    $$$ASSERT('..Adapter.Exists(tFilename))
    Set tSC=..Adapter.PutStream(tFilename,pRequest.StreamIn) Quit:$$$ISERR(tSC) tSC
    Quit $$$OK
  }
}

```

More complicated scenarios are sometimes useful, such as for instance executing a number of SQL statements in a business operation using the SQL adapter and then calling a rollback before returning if any of them fail. Depending on the API being called, it may be necessary to check public variables in addition to any returned status values. An example of this is checking SQLCODE in the case of embedded SQL. However, the coding style in the previous example is the best practice in simple circumstances.

2.3 Passing Values by Reference or as Output

If you are not familiar with passing values by reference or output, this section is intended to orient you to this practice.

Many InterSystems IRIS methods return at least two values: a status (an instance of %Status) *and* a response message or other returned value. Typically the response message is returned by reference or as output. If a value is returned by reference or as output, that means:

- When you define the method, the method must set the corresponding variable.
- When you invoke the method, you must include a period before the corresponding argument.

The following examples demonstrate these points.

2.3.1 Typical Callback Method

The following shows the signature of a typical callback method:

```
method OnRequest(request As %Library.Persistent, Output response As %Library.Persistent) as %Status {}
```

The keyword `Output` indicates that the second argument is meant to be returned as output. In your implementation of this method, you would need to do the following tasks, in order to satisfy the method signature:

1. Set a variable named `response` equal to an appropriate value. This variable must have a value when the method completes execution.
2. End with the `Quit` command, followed by the name of a variable that refers to an instance of %Status.

For example:

Class Member

```
Method OnRequest(request As %Library.Persistent, Output response As %Library.Persistent) as %Status
{
    //other stuff
    set response=myObject
    set pSC=..MyMethod() ; returns a status code
    quit pSC
}
```

Note: When setting the response equal to the request, be sure to use **%ConstructClone()**. Otherwise, the request object will change as you manipulate the response, giving you an inaccurate record of the message that was sent to the business host. For example, if you want to set the response to the request, enter:

Class Member

```
Method OnRequest(request As %Library.Persistent, Output response As %Library.Persistent) as %Status
{
    set response=request.%ConstructClone()
    // manipulate response without affecting the request object
}
```

This example discusses a value returned as output, but the details are the same for a value passed by reference.

2.3.2 Typical Helper Method

The following shows the signature of a typical inherited helper method:

Class Member

```
method SendRequestSync(pTargetDispatchName As %String,
    pRequest As %Library.Persistent,
    ByRef pResponse As %Library.Persistent,
    pTimeout As %Numeric = -1,
    pDescription As %String = "") as %Status {}
```

The keyword **ByRef** indicates that the third argument is meant to be returned by reference. To invoke this method, you would use the following:

ObjectScript

```
set sc=##class(pkg.class).SendRequestSync(target,request,.response,timeout,description)
```

Notice the period before the third argument.

This example discusses a value passed by reference, but the details are the same for a value returned as output.

2.4 Accessing Properties and Methods from a Business Host

When you define a method in a business host class, you might need to access properties or methods of that class or of the associated adapter. This section briefly describes how to do these things.

Within an instance method in a business host, you can use the following syntaxes:

- `..bushostproperty`

Accesses a setting or any other property of the business host. (Remember that all settings are properties of their respective classes.)

- `..bushostmethod()`

Accesses an instance method of the business host.

- `..Adapter.adapterproperty`

Accesses a setting or any other property of the adapter. (Note that every business host has the property `Adapter`. Use that property to access the adapter and then use dot syntax to access properties of the adapter.)

- `..Adapter.adaptermethod()`

Accesses an instance method of the adapter, passing in arguments to the method. For example, to invoke the `PutStream` method of an outbound adapter from a business operation, enter:

```
..Adapter.PutStream(pFilename, ..%TempStream)
```

2.5 Accessing Production Settings

You might need to access a setting of the production. To do so, use the macro `$$$ConfigProdSetting`. For example, `$$$ConfigProdSetting("mySetting")` retrieves the value of the production setting called `mySetting`. InterSystems suggests you wrap this macro in a `$GET` call for safety; for example:

```
set myvalue=$GET($$$$ConfigProdSetting("mySetting"))
```

Also see [Using Ens.Director to Access Settings](#).

2.6 Choosing How to Send Messages

In business operations and business processes, your methods typically invoke inherited methods to send messages to other business hosts within the production. This section discusses the options.

2.6.1 Synchronous and Asynchronous Sending

When you define business service, business process, and business operation classes, you specify how to send a request message from that business host. There are two primary options:

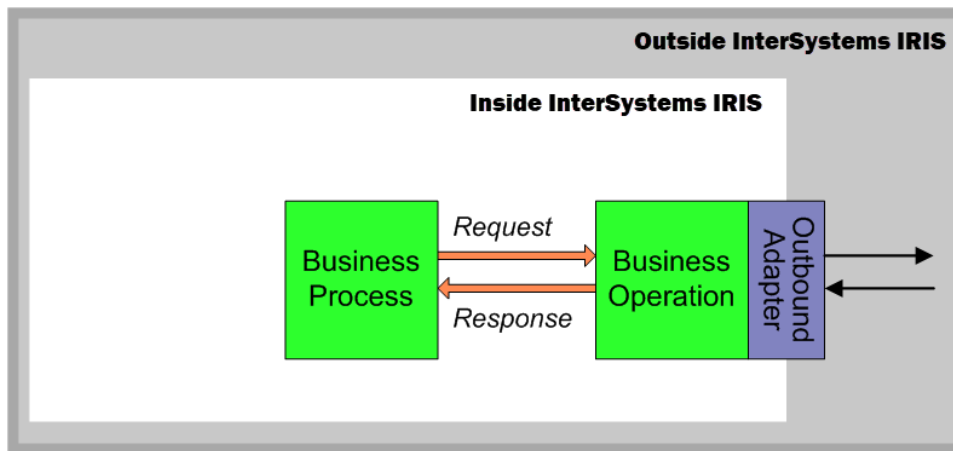
- *Synchronously*—The caller stops all processing to wait for the response.
- *Asynchronously*—The caller does not wait; immediately after sending the request the caller resumes other processing. When sending a request asynchronously, the caller specifies one of two options regarding the response to this request:
 - Ask to receive the response when it arrives.
 - Ignore the possibility of a response.

The choice of how to send a message is *not* recorded in the message itself and is not part of the definition of the message. Instead, this is determined by the business host class that sends the message.

2.6.2 Deferred Sending

In addition to the straightforward alternatives of synchronous (wait) and asynchronous (do not wait), it is possible to send messages outside InterSystems IRIS using a mechanism called *deferred response*.

Suppose a business process wishes to invoke an action outside InterSystems IRIS. It sends a request to a business operation, which performs the invocation and returns the response. The business process is the intended recipient of any response; the business operation is simply the means by which the request goes out and the response comes in. The business operation will relay a response back if the business process made the request synchronously, or if it made the request asynchronously with asynchronous response requested. The following diagram summarizes this mechanism.

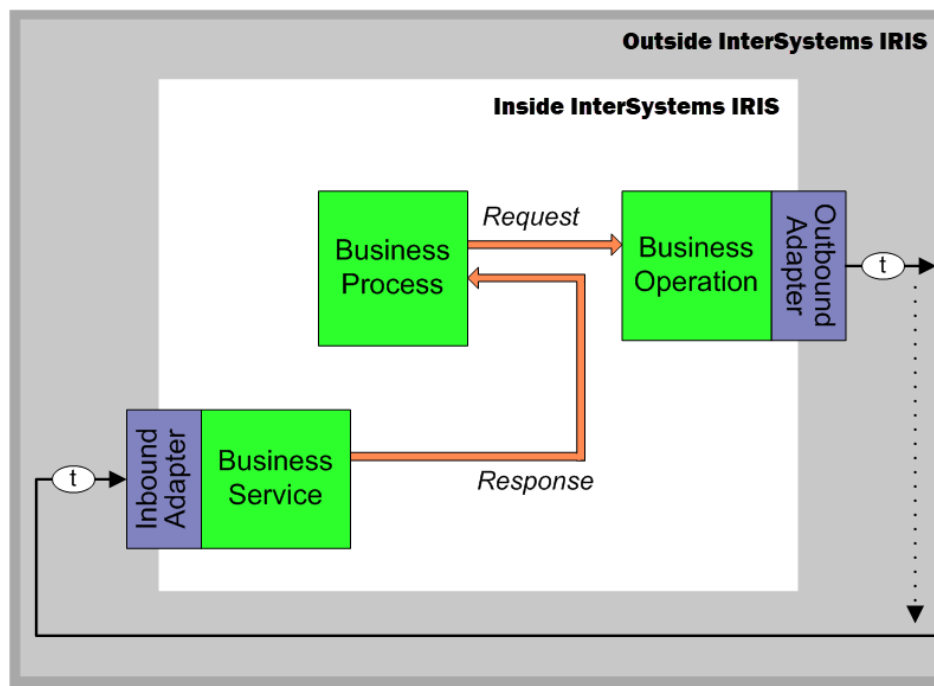


Now suppose the business operation that receives a request from a business process has been written to use the deferred response feature. The original sender is unaware of the fact that the response is going to be deferred by the business operation. Deferring the response is a design decision made by the developer of the business operation. If the business operation does in fact defer the response, when the original sender receives the response at the end of the deferral period, it is unaware that the response was ever deferred.

A business operation defers a response by calling its **DeferResponse()** method to generate a token that represents the original sender and the original request. The business operation must also find a way to communicate this token to the external entity, which is then responsible for including this token in any later responses to InterSystems IRIS. For example, if the external destination is email, a business operation can include the token string in the subject line of the outgoing email. The entity receiving this email can extract this token from the request subject line and use it in the response subject line. In the following diagram, the item *t* represents this token.

Between the time when the business operation defers the request, and when the response is finally received by the original sender, the request message has a status of Deferred. After the original sender receives the corresponding response, the request message status changes from Deferred to Completed.

An incoming event in response to the request can be picked up and returned to the original sender by any business host in the production. Exactly where the event arrives in an InterSystems IRIS production depends on the design of the production; typically, it is the task of a business service to receive incoming events from outside InterSystems IRIS. The business host that receives the incoming event must also receive the deferred response token with the event. The business host then calls its **SendDeferredResponse()** method to create the appropriate response message from the incoming event data and direct this response to the original sender. The original sender receives the response without any knowledge of how it was returned. The following figure shows a request and its deferred response.



2.7 Generating Alerts

An *alert* sends notifications to applicable users while a production is running, in the event that an alert event occurs. The intention is to alert a system administrator or service technician to the presence of a problem. Alerts may be delivered via email or other mechanism. All alerts *also* write messages to the InterSystems IRIS [Event Log](#), with the type Alert.

The production alert mechanism works as follows:

- When you create business host classes for the production, include code that:
 1. Detects undesirable conditions or other circumstances that a user must address.
 2. Generates alerts on those occasions.
- You define and configure an *alert processor*, which is a business host, named Ens.Alert. The alert processor can optionally manage the alert to track the process of resolving the event. For details on defining an alert processor, see [Defining an Alert Processor](#). Any production can include no more than one alert processor.

In a business host class (other than a BPL process class), do the following to generate an alert:

1. Create an instance of Ens.AlertRequest.
2. Set the AlertText property of this instance. Specify it as a string that provides enough information so that the technician has a good idea of how to address the problem.
3. Invoke the **SendAlert()** method of the business host class. This method runs asynchronously and thus does not delay the normal activities of the business host.

Note: For information on generating alerts in BPL, see [Developing BPL Processes](#).

2.8 See Also

- [Adding Logging and Tracing](#)
- [Adding and Removing Settings](#)

3

Adding Logging and Tracing

This page describes how to add Event Log entries and trace messages to a production. The Event Log is intended primarily for system administrators, but which is also useful during development. Trace messages, in contrast, are primarily for use during development.

3.1 Generating Event Log Entries

The Event Log is a table that records events that have occurred in the production running in a given namespace. The Management Portal provides a [page](#) that displays this log, which is intended primarily for system administrators, but which is also useful during development.

The primary purpose of the Event Log is to provide diagnostic information that would be useful to a system administrator in case of a problem while the production is running.

InterSystems IRIS automatically generates Event Log entries, and you can add your own entries. Any given event is one of the following types: Assert, Info, Warning, Error, and Status. (The Event Log can also include [alert messages](#) and [trace items](#).)

To generate Event Log entries:

1. Identify the events to log.

Not all types of error or activity should necessarily generate Event Log entries. You must choose the occurrences to note, the type to use, and the information to record. For example, Event Log entries should appear in case of an external, physical problem, such as a bad network connection.

The Event Log should not register program errors; these should be resolved before the production is released.

2. Modify the applicable parts of the production (typically business host classes) to generate Event Log entries in [ObjectScript](#), as described in the following subsection.

Important: If you need to notify users actively about certain conditions or events, use alerts, which are discussed in [Generating Alerts](#) and in [Defining Alert Processors](#).

3.1.1 Generating Event Log Entries in ObjectScript

Within business host classes or other code used by a production, you can generate Event Log entries in ObjectScript. To do so, use any of the following macros. These macros are defined in the `Ensemble.inc` include file, which is automatically included in InterSystems IRIS system classes:

Macro	Details
<code>\$\$\$LOGINFO(<i>message</i>)</code>	Writes an entry of type Info. Here and later in this table, <i>message</i> is a string literal or an ObjectScript expression that evaluates to a string.
<code>\$\$\$LOGERROR(<i>message</i>)</code>	Writes an entry of type Error.
<code>\$\$\$LOGWARNING(<i>message</i>)</code>	Writes an entry of type Warning.
<code>\$\$\$LOGSTATUS(<i>status_code</i>)</code>	Writes an entry of type Error or Info, depending on the value of the given <i>status_code</i> , which must be an instance of %Status.
<code>\$\$\$ASSERT(<i>condition</i>)</code>	Writes an entry of type Assert, if the argument is false. <i>condition</i> is an ObjectScript expression that evaluates to true or false.
<code>\$\$\$LOGASSERT(<i>condition</i>)</code>	Writes an entry of type Assert, for any value of the argument. <i>condition</i> is an ObjectScript expression that evaluates to true or false.

The following shows an example with an expression that combines static text with the values of class properties:

ObjectScript

```
$$$LOGERROR("Awaiting connect on port "_.Port_" with timeout "_.CallInterval)
```

The following example uses an ObjectScript function:

ObjectScript

```
$$$LOGINFO("Got data chunk, size="_$length(data)_"_"_tChunkSize)
```

3.2 Adding Trace Elements

Tracing is a tool for use primarily during development. You add trace elements so that you can see the behavior of various elements in a production, for the purpose of debugging or diagnosis. To add trace elements to a production, you identify the areas in your code (typically business host classes) where you would like to see runtime information. In those areas, you add lines of code that (potentially) write trace messages. Note that these are messages only in a general sense; trace messages are simply strings and are unrelated to `Ens.Message` and its subclasses.

Tracing consists of two parts:

- Writing trace messages in applicable parts of the production (as described here)
- Enabling tracing as described in [Enabling Tracing](#).

3.2.1 Writing Trace Messages in ObjectScript

To write trace messages in ObjectScript, use the following lines of code:

- To write a user trace message:

ObjectScript

```
$$$TRACE(trace_message)
```

Where *trace_message* is a string containing useful information about the context in which you add this line of code.

- To write a system trace message (less common):

ObjectScript

```
$$$sysTRACE(trace_message)
```

For example:

ObjectScript

```
$$$TRACE("received application for "_request.CustomerName)
```

3.2.2 Writing Trace Messages from BPL or DTL

To write trace messages in a BPL business process, use the [BPL <trace>](#) element.

To write trace messages in a DTL data transformation, use the [DTL <trace>](#) element.

3.3 See Also

- [Using the Event Log](#)

4

Adding and Removing Settings

This page describes how to add settings to and remove settings from elements of a production.

4.1 Adding Settings

To provide new settings for a production, a business host, or an adapter, modify its class definition as follows:

1. Add a property for each configuration setting you wish to define.
2. Add a class parameter called *SETTINGS* to the class.
3. Set the value of *SETTINGS* to be a comma-delimited list of the names of the properties you have just defined. For example:

```
Property foo As %String;  
Property bar As %String;  
Parameter SETTINGS = "foo,bar";
```

See the [following section](#) for additional details for *SETTINGS*.

The `foo` and `bar` settings now automatically appear in the configuration display on the **Production Configuration** page whenever an item of that class is selected for configuration.

It is possible to have a property with the same name within an adapter class and within a business host class that uses the given adapter. It is also possible to expose both of these properties as settings. The Production Configuration page displays these settings separately with separate tool tips generated by the property definitions in the respective classes.

4.2 Removing Settings

To remove an inherited configuration setting, for a production, a business host, or an adapter, modify its class definition as follows. Update the *SETTINGS* class parameter of that class to include the setting name, preceded by a hyphen (-). For example:

Class Member

```
Parameter SETTINGS = "-foo";
```

Note that by design, some settings cannot be removed; for example, the Pool Size setting cannot be removed.

4.3 Specifying Categories and Controls

By default, a setting is displayed in the **Additional Settings** category on the **Details** tab when you configure a production. By default, the setting provides one of the following input mechanisms, depending on the property on which the setting is based:

- In most cases, the setting is displayed with a plain input field.
- If the property specifies the *VALUELIST* parameter, the setting is displayed with a drop-down list. The items that will appear in the drop-down list must be separated using a comma as the delimiter.
- If the property is of type *%Boolean*, the setting is displayed as a check box.

In all cases, the *InitialExpression* of the property (if specified) controls the initial state of the input field.

You can override both the location and control type. To do so, include the setting name in *SETTINGS* as follows:

Class Member

```
Parameter SETTINGS = "somesetting:category:control";
```

Or:

Class Member

```
Parameter SETTINGS = "somesetting:category";
```

Class Member

```
Parameter SETTINGS = "somesetting::control";
```

Where *category* and *control* indicate the [category](#) and [control](#) to use, respectively. The following subsections provide details.

You can include multiple settings, as follows:

Class Member

```
Parameter SETTINGS = "setting1:category:control1,setting2:control2:editor,setting3:category:control3";
```

For example (with disallowed line breaks included):

```
Parameter SETTINGS = "HTTPServer:Basic,HTTPPort:Basic,SSLConfig:Connection:sslConfigSelector,  
ProxyServer:Connection,ProxyPort:Connection,ProxyHTTPS:Connection,  
URL:Basic,Credentials:Basic:credentialsSelector,UseCookies,ResponseTimeout:Connection"
```

4.4 Available Categories

Within the [SETTINGS](#) parameter, *category* is one of the following case-sensitive, literal values:

- **Info**—Places the setting into the **Information Settings** category.
- **Basic**—Places the setting into the **Basic Settings** category.
- **Connection**—Places the setting into the **Connection Settings** category.

- **Additional**—Places the setting into the **Additional Settings** category (the default).
- **Alerting**—Places the setting into the **Alerting Control** category.
- **Dev**—Places the setting into the **Development and Debugging** category.

Or use your own category name.

4.5 Available Controls

Within the [SETTINGS](#) parameter, *control* specifies the name of a specific control to use when viewing and modifying the setting in the **Production Configuration** page. Use `selector` or use the name of a class in the package `EnsPortal.Component`. See [Examples for Settings Controls](#).

To supply extra values to your component in order to display a suitable set of options, append a set of name-value pairs as follows:

```
myControl?Prop1=Value1&Prop2=Value2&Prop3=Value3
```

Where:

- *Prop1*, *Prop2*, *Prop3*, and so on are names of properties of the control. For example: `multiSelect`
- *Value1*, *Value2*, *Value3*, and so on are the corresponding values.

Use the syntax `@propertyname` to refer to a property of the JavaScript `zenPage` object. The supported variables are these:

- *currHostId* refers to the ID of the current `Ens.Config.Item`.
- *productionId* refers to the ID of the `Ens.Config.Production` in use.

Use `\@` to indicate the character `@` as is.

To pass values to the context property of a control, enclose the value in curly braces. See the [subsection](#).

4.5.1 The selector Control

InterSystems IRIS uses the general-purpose selector component, which you can also use; to do so, specify *control* as `selector` and then append property/value pairs.

The `selector` component allows almost any list of data or options to be displayed to the user and which allows the user to type data if needed. To configure the component in this way, use the following syntax to set the context property of the control:

```
context={ContextClass/ContextMethod?Arg1=Value1&Arg2=Value2&Arg3=Value3}
```

Where:

- *ContextClass* is the name of the context class. This must be a subclass of `%ZEN.Portal.ContextSearch` or `Ens.ContextSearch`. See [Examples for Settings Controls](#).
- *ContextMethod* is a method in that class. This method provides the data from which the user chooses values.
- *Arg1*, *Arg2*, *Arg3*, and so on are the names of multidimensional arguments of that method.
- *Value1*, *Value2*, *Value3*, and so on are values for those arguments. The rules for values are the same as given previously (for example, you can use `@propertyname`).

Note: Note that the selector component also has a property named `multiSelect`. By default, the user can select only one item. To enable the user to select multiple items, include a property/value pair as follows: `multiSelect=1`.

The argument names and values must be URL encoded. For example, replace `%` with `%25`.

4.5.2 Examples for Settings Controls

The following list gives examples for *control*, organized by the kind of data that they enable the user to select.

These examples use InterSystems IRIS classes. Many of these examples use the `Ens.ContextSearch` class, which provides a large number of useful methods. If this list does not cover your scenario, see the class documentation for `Ens.ContextSearch` to determine whether any of the existing methods supply the data you need. If that class does not cover your scenario, you can create your own subclass of `Ens.ContextSearch`.

BPL

`bplSelector`

business hosts in the same production

`selector?multiSelect=1&context={Ens.ContextSearch/ProductionItems?targets=1&productionName=@productionId}`

Or, if the user should choose only one business host:

`selector?context={Ens.ContextSearch/ProductionItems?targets=1&productionName=@productionId}`

business partners

`partnerSelector`

business rules

`ruleSelector`

character sets

`selector?context={Ens.ContextSearch/CharacterSets}`

credential sets

`credentialsSelector`

directories

`directorySelector`

DTL

`dtlSelector`

files

`fileSelector`

framing

`selector?context={Ens.ContextSearch/getDisplayList?host=@currHostId&prop=Framing}`

local interfaces

```
selector?context={Ens.ContextSearch/TCPLocalInterfaces}
```

schema categories appropriate for a specific business host

```
selector?context={Ens.ContextSearch/SchemaCategories?host=classname}
```

Where *classname* is the class name of the business host. For example:

```
selector?context={Ens.ContextSearch/SearchTableClasses?host=EnsLib.MsgRouter.RoutingEngineST}
```

schedules

```
scheduleSelector
```

search table classes appropriate for a specific business host

```
selector?context={Ens.ContextSearch/SearchTableClasses?host=classname}
```

Where *classname* is the class name of the business host. For example:

```
selector?context={Ens.ContextSearch/SearchTableClasses?host=EnsLib.EDI.EDIFACT.Service.Standard}
```

SSL configurations

```
sslConfigSelector
```

4.6 Specifying Default Values

You should consider how to control the default values for any settings of those items. InterSystems IRIS can take the default value for a setting from one of three sources:

- The production definition.
- The values defined for the InterSystems IRIS instance, but stored outside the production. For information, see [Defining Production Defaults](#).
- The default value for the property as defined in the host class. In this case, the default value is determined by the `InitialExpression` property keyword.

Some settings are dependent on the environment, such as TCP/IP addresses or file paths; typically you configure these settings to have their source outside the production, while others, such as `ReplyCodeActions` are design decisions and most likely you develop your application to retrieve these from the production definition.

You can develop your production to have configuration settings come from different sources. The primary purpose is to make it easier to move productions from one InterSystems IRIS instance to another, such as from *test* to *live*.

Once you define a production, you can change the source of both production and business host settings on the **Production Configuration** page of the Management Portal; see [Configuring Settings](#).

The use of default settings allows you to define production and business host settings outside the production definition where you can preserve them during a production upgrade. To facilitate updating productions or moving productions from one system to another, you can omit settings and take their values from a structure that is installed on the system. When a setting is missing, InterSystems IRIS retrieves the default setting from outside the production definition if one exists.

For programming details, see `Ens.Director` class for the method signatures for the following methods:

- **GetProductionSettingValue()**
- **GetProductionSettings()**

4.7 Considerations When Redefining Settings

Once you have exposed a property value as a setting, changing the name of that property poses difficulties where the setting has been used. If you change the name of such a property, it may also be necessary to change the following items:

- Existing productions may specify a value for the setting. Because the original property no longer exists, the Production Configuration page no longer allows users to modify the value. Instead you must either directly edit the XData block in the production class (via an IDE) or remove the value programmatically. To remove the value programmatically, use the `Ens.Director` class, which provides the following methods for this purpose:
 - **SetHostSettingValue()** for business hosts
 - **SetAdapterSettingValue()** for adapters
 - **SetItemSettingValue()** for business hosts or adapters

If the value is not removed, the system logs a message when the production is started.

- If System Default Setting used to specify the value for the setting, then the System Default Setting entry must be edited.
- Any custom code that refers to the property must be changed.

5

Defining Messages

This page describes how to define classes to contain message bodies for an InterSystems IRIS® data platform interoperability production.

5.1 Types of Message Bodies

A message body class must be a persistent class; messages sent by a production are all stored in the database until a system administrator purges them. There are two general types of message body classes, *standard message bodies* and *virtual documents*. The distinction is that a standard message body contains multiple properties that contain the message data, and a virtual document stores the complete message data in serialized form. Standard message bodies are suitable when the messages always have the same form. When the data is arbitrarily long or complex, it is necessary to use virtual documents instead. For Electronic Data Interchange (EDI) formats, InterSystems IRIS uses virtual documents.

In either case, InterSystems IRIS provides tools to enable you to access specific elements within the message, for the purposes of data transformation, routing, or business rules.

A standard message body can be [simple](#) (containing only non-object properties) or can be [complex](#) (containing object-valued properties).

For information on virtual documents, see [Using Virtual Documents in Productions](#). The rest of this page discusses only standard message bodies.

5.2 Creating a Simple Message Body Class

To create a simple standard message class, create a class that:

- Extends %Persistent.

Note that in general it is important to consider how the messages will be stored, in particular to make sure that messages of different types are stored in different tables, for best performance when searching messages. See [Message Storage](#), which also describes alternative approaches to creating a simple standard message class.

- Also extends either `Ens.Util.RequestBodyMethods` or `Ens.Util.ResponseBodyMethods`, depending on whether the class is to be used as a request message or a response message, respectively. If you use these classes, you have easy access to the various built-in features for viewing the contents of messages from the Management Portal. These features help developers and administrators detect errors in a running production, especially if the production uses message content to determine where the message should be sent.

- Optionally also extends %XML.Adaptor. In this case, the Management Portal can display the full message in XML format. Also see [Controlling the Display](#).
- Contains properties as needed to represent elements of data to be carried in the message.
- Optionally specifies a value for the ENSPURGE parameter; see [Specifying Message Purge Behavior](#).

The following shows a simple example:

Class Definition

```
Class Demo.Loan.Msg.CreditRatingResponse Extends (%Persistent, Ens.Util.ResponseBodyMethods)
{
Property TaxID As %String;
Property CreditRating As %Integer;
}
```

The class can also contain methods. For example:

Class Definition

```
Class Demo.Loan.Msg.Application Extends (%Persistent, Ens.Util.RequestBodyMethods)
{
Property Amount As %Integer;
Property Name As %String;
Property TaxID As %String;
Property Nationality As %String;
Property BusinessOperationType As %String;
Property Destination As %String;
Method RecordNumber() As %String
{
  If ..%Id()="" Do ..%Save()
  Quit ..%Id()
}
Method GetRecordNumberText(pFormatAsHTML As %Boolean = 0) As %String
{
  Set tCRLF=$$(pFormatAsHTML:"<br>",1:$c(13,10))
  Set tText=""
  Set tText=tText_"Your loan application has been received,"_tCRLF
  Set tText=tText_"and is being processed."_tCRLF
  Set tText=tText_"Your record number is "_.RecordNumber()_"_tCRLF
  Set tText=tText_"You'll receive a reply from FindRate"_tCRLF
  Set tText=tText_"within 2 business days."_tCRLF
  Set tText=tText_"Thank you for applying with FindRate."_tCRLF
  Quit tText
}
}
```

5.3 Message Storage

It is important to store messages of different types in different globals, for best performance when searching messages. To achieve this goal, you can use any of the following approaches:

- A message class can extend %Persistent and Ens.Util.RequestBodyMethods or Ens.Util.ResponseBodyMethods, as described previously.
- A message class can extend %Persistent and then Ens.Request or Ens.Response.

Important: When using multiple persistent superclasses, as in this case, the order of superclasses is important. If `Ens.Request` or `Ens.Response` is listed first, the messages will be saved in the same global that stores all other requests or responses, and this can adversely affect performance when querying the message tables. In contrast, if `%Persistent` is listed first, the messages are saved in their own global and their own table.

- A message class can extend any persistent class and can use the `USEEXTENTSET` and `DEFAULTGLOBAL` class parameters, which provide other ways of ensuring that the data is stored in its own global and its own table; for details, see the class reference for `%Persistent`.

5.4 Controlling the Display

In the Management Portal, both the Message Viewer and the Visual Trace provide two tabs that display the message body:

- The **Body** tab displays the message body as a table, with one column representing properties of the message class, and the second column displaying the corresponding values. This table does not display any object-valued properties.

By default, this table cannot display more than 99 properties. To increase the number of properties that it can display, enter the following command in the ObjectScript shell in the namespace where this maximum should be increased:

ObjectScript

```
set ^CSP.AutoFormMaxProperties=newmaximum
```

Where *newmaximum* is an integer that specifies the maximum number of properties to display. (This change also affects the [testing service input form form](#).)

- The **Contents** tab displays the message body in serialized form, if possible. If the message class inherits from `%XML.Adaptor`, this tab displays the message in XML format.

If you want to override this display, you can do so by defining `%GetContentType()` and `%ShowContents()` in your message class. For information, see the class reference for `Ens.Util.MessageBodyMethods`.

5.5 Creating a Complex Message Body Class

In the previous example, the message body class contained only simple properties. In some cases, you may need to define properties that use other classes. If so, you should carefully consider what to do when you purge message bodies (as described in [Purging Data](#)).

When you purge message bodies, InterSystems IRIS deletes only the specific message body object. For example, consider the following message class:

Class Definition

```
Class MyApp.Messages.Person Extends Ens.Util.RequestBodyMethods
{
Property Name As %String;
Property MRN As %String;
Property BirthDate As %Date;
Property Address As MyApp.Messages.Address;
}
```

The Address class is as follows:

Class Definition

```
Class MyApp.Messages.Address Extends %Persistent
{
Property StreetAddress As %String;
Property City As %String;
Property State As %String;
Property ZIP As %String;
}
```

In this case, if you purge message bodies, InterSystems IRIS deletes instances of `MyApp.Messages.Person`, but does not delete instances of `MyApp.Messages.Address`.

If your message body class uses other classes as properties and if your application requires that any referenced objects should also be purged, use one of the following approaches:

- Make sure that the referenced classes are serial. For example, redefine the Address class as follows:

```
Class MyApp.Messages.Address Extends %SerialObject
{
...
}
```

In this case, the data for the Address class is stored as part of the Person class (and is thus automatically purged at the same time).

- Define the property as a suitable relationship. See Relationships.
- Add a delete trigger or **%OnDelete()** method to the message class so that this class deletes the appropriate records in the referenced classes.
- Optionally include `%XML.Adaptor` as a superclass so that the properties defined in the referenced class can be displayed in the Management Portal.

5.6 Specifying Message Purge Behavior

When you define a message body class, you can include the `ENSPURGE` parameter to specify how InterSystems IRIS handles instances of the class during purge operations. The parameter has two possible values:

- 0—InterSystems IRIS does not purge message bodies based on the class, even when the option to purge message bodies is enabled.
- 1—InterSystems IRIS purges message bodies based on the class when the option to purge message bodies is enabled.

The ENSPURGE parameter affects all purges from the Management Portal, except for purges of the [Enterprise Message Bank](#). Similarly, it affects programmatic purges using the **Purge()** method of the `Ens.MessageHeader` class.

For example, consider the `Sample.Person` persistent database class:

Class Definition

```
Class Sample.Person Extends (%Persistent, %Populate, %XML.Adaptor)
{
  Property Name As %String(POPSPEC = "Name()") [ Required ];
  Property SSN As %String(PATTERN = "3N1"-""2N1"-""4N") [ Required ];
  Property DOB As %Date(POPSPEC = "Date()");
  //...
}
```

If you configure a production to send `Sample.Person` objects to a business operation that updates patient information, it might be important to retain the objects. To ensure that the system does not purge any instances of the `Sample.Person` message body class, you could add the ENSPURGE parameter to the class definition as follows:

Class Definition

```
Class Sample.Person Extends (%Persistent, %Populate, %XML.Adaptor)
{
  Parameter ENSPURGE As %Boolean = 0;
  Property Name As %String(POPSPEC = "Name()") [ Required ];
  Property SSN As %String(PATTERN = "3N1"-""2N1"-""4N") [ Required ];
  Property DOB As %Date(POPSPEC = "Date()");
  //...
}
```

During subsequent purges, the system removes only the headers of messages based on the `Sample.Person` message body class. The message bodies are essentially orphaned and can be removed only programmatically. For more information, see [Purging Production Data](#).

The ENSPURGE parameter is inheritable and is not required. The default value is 1.

5.7 See Also

- [Using Virtual Documents in Productions](#)
- [Purging Production Data](#)

6

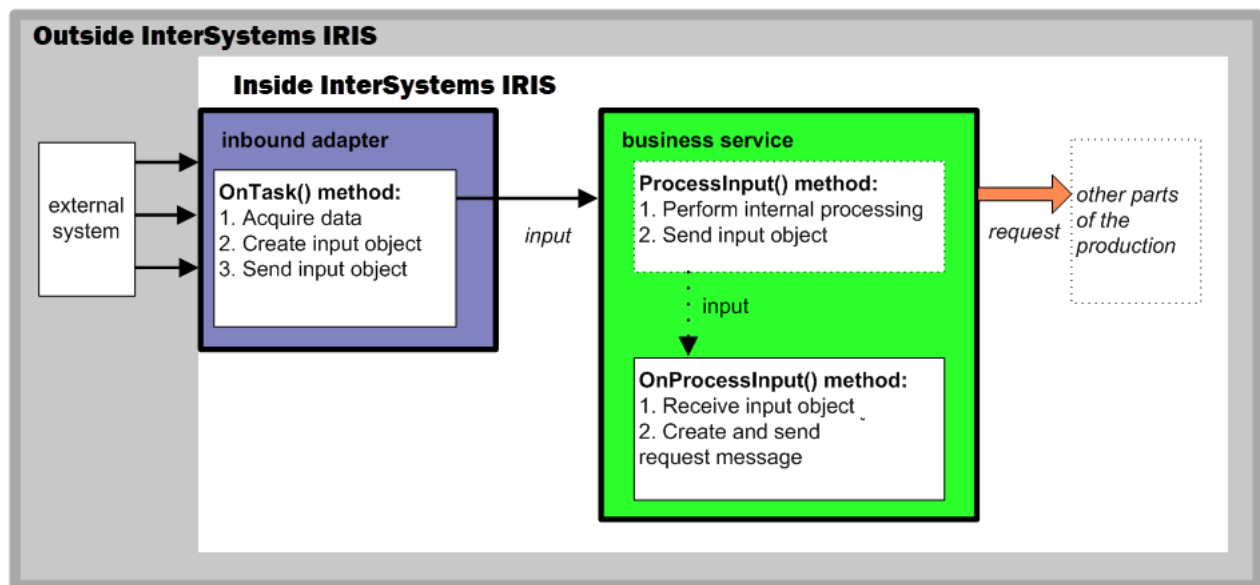
Defining Business Services

This page describes how to define business service classes.

Tip: InterSystems IRIS® data platform provides specialized business service classes that use specific inbound adapters, and one of those might be suitable for your needs. If so, no programming would be needed. For a partial list, see [Connectivity Options](#).

6.1 Introduction to Defining Business Services

A business service is responsible for accepting requests from external applications into InterSystems IRIS. The following figure shows how it works:



Note that this figure shows only the input flow of data, not the optional response.

A business service is responsible for the following activities:

- Waiting for a specific external event (such as notification from an application, receipt of a TCP message, etc.).
- Reading, parsing, and validating the data accompanying such an event,
- Returning, if required, an acknowledgment to the external application indicating that the event was received.

- Creating an instance of a request message and forwarding it on to the appropriate business process or business operation for processing.

The purpose of a business service is usually to receive data input. In most cases, a business service has an inbound adapter associated with it. However, in some cases an adapter is not required, either because an application is capable of sending request messages into the service or because the business service has been written to handle external calls of a particular kind, for example from a composite application. A business service of this type is called an *adapterless* business service.

When a business service has an inbound adapter, it is in the data pulling (as opposed to pushing) mode. In this mode, the business service polls the adapter at regular intervals to see if it has data. Meanwhile, if the adapter encounters input data at any time, it calls the business service to process the input.

When a business service does not have an adapter, it does not pull data. Instead, client applications call the business service and tell it to process input (this is a data pushing mode).

6.2 Key Principles for Business Services

First, be sure to read [Programming in InterSystems IRIS](#).

Within a business service, you can access properties and methods of the associated adapter, which is available as the Adapter property of the business service. This means that you can alter the default behavior of the adapter; it may or may not be appropriate to do so. It is useful to remember the principle of encapsulation. The idea of encapsulation is that the adapter class should be responsible for the technology-specific logic, while the business service class should be responsible for the production-specific logic.

If you find that it is necessary to greatly or frequently alter the behavior of an adapter class from within a business service class, it might be more appropriate to create a customized subclass of the adapter class. See [Less Common Tasks](#).

This principle also applies to business operations.

6.3 Defining a Business Service Class

To create a business service class, define a class as follows:

- The class must extend `Ens.BusinessService` (or a subclass).
- The *ADAPTER* parameter must equal the name of the adapter class for this business service to use.

Tip: If you simply want a business service to wake up and run periodically without concern for events outside InterSystems IRIS, use the adapter class `Ens.InboundAdapter`.

- The class must implement the `OnProcessInput()` method, as described in [Implementing the OnProcessInput\(\) Method](#).
- The class can add or remove settings. See [Adding and Removing Settings](#).
- The class can implement any or all of the startup and teardown methods. See [Overriding Start and Stop Behavior](#).
- The class can contain methods to accomplish work internal to itself.

For examples of business service classes, see the adapter guides.

6.4 Implementing the OnProcessInput() Method

Within your business service class, your **OnProcessInput()** method should have the following generic signature:

Class Member

```
Method OnProcessInput(pInput As %RegisteredObject, Output pOutput As %RegisteredObject) As %Status {}
```

Here *pInput* is the input object that the adapter will send to this business service, and *pOutput* is the output object.

First look at the adapter class that you have selected. InterSystems recommends that you edit the **OnProcessInput()** method signature to use the specific input argument needed with the adapter.

The **OnProcessInput()** method should do some or all of the following:

1. Optionally set properties of the business service class (at any appropriate time). The business service property of greatest interest is `%WaitForNextCallInterval`. Its value controls how frequently InterSystems IRIS invokes the **OnTask()** method of the adapter.

For other properties, see the class reference for `Ens.BusinessService`.

2. Validate, if necessary, the input object.
3. Examine the input object and decide how to use it.
4. Create an instance of a request message class, which will be the message that your business service sends.
For information on creating messages, see [Creating Messages](#).
5. For the request message, set its properties as appropriate, using values in the input object.
6. Determine where you want to send the request message. When you send the message, you will need to use the configuration name of a business host within the production.
7. Send the request message to a destination within the production (a business process or business operation). See the [next section](#).
8. Make sure that you set the output argument (*pOutput*). Typically you set this equal to the response message that you have received. This step is required.
9. Return an appropriate status. This step is required.

6.5 Sending Request Messages

In your business service class, your implementation of **OnProcessInput()** should send a request message to some destination within the production. To do so, call one of the following instance methods of the business service class, as appropriate for your needs:

- **SendRequestSync()** sends a message synchronously (waits for a response). For details, see [Using the SendRequestSync\(\) Method](#).
- **SendRequestAsync()** sends a message asynchronously (does not wait for a response). For details, see [Using the SendRequestAsync\(\) Method](#).
- **SendDeferredResponse()** sends a response that was previously deferred. This method is less commonly used. For details, see [Using the SendDeferredResponse\(\) Method](#).

Each of these methods returns a status, an instance of %Status.

These methods are also defined—with the identical method signatures—in `Ens.BusinessProcess` and `Ens.BusinessOperation`, although their internals are different in those classes. This means that you can invoke these instance methods from within your business process and business operation classes.

6.5.1 Using the `SendRequestSync()` Method

To send a synchronous request, use the **`SendRequestSync()`** method as follows:

ObjectScript

```
Set tSC = ..SendRequestSync(pTargetDispatchName, pRequest, .pResponse, pTimeout)
```

Where:

- *pTargetDispatchName*—The configuration name of the business process or business operation to which the request is sent.
- *pRequest*—A request message. See [Defining Messages](#).
- *pResponse*—(By reference) A response message. This object receives the data returned by the response.
- *pTimeout*—(Optional) The number of seconds to wait for a response. The default is -1 (wait forever).

This method returns a status, an instance of %Status.

If no response is expected, you can use **`SendRequestAsync()`** instead of **`SendRequestSync()`**.

6.5.2 Using the `SendRequestAsync()` Method

To send an asynchronous request, use the **`SendRequestAsync()`** method as follows:

ObjectScript

```
Set tSC = ..SendRequestAsync(pTargetDispatchName, pRequest)
```

Where:

- *pTargetDispatchName*—The configuration name of the business process or business operation to which the request is sent.
- *pRequest*—A request message. See [Defining Messages](#).

This method returns a status, an instance of %Status.

If you use **`SendRequestAsync()`** to send a message to a business process and the business process sets the response to a persistent object, the response is saved without a corresponding response message header. This happens whenever the requestor does not ask for a response and the business process creates the response. (If this is not what you want, have the [business process](#) set the response to null.)

6.5.3 Using the `SendDeferredResponse()` Method

All business hosts support the **`SendDeferredResponse()`** method. This method permits a business host to participate in the production deferred response mechanism. The business host identifies a previously deferred request, creates the actual response message, and sends this response to the business host that originated the request. See [Using Deferred Sending in Programming in InterSystems IRIS](#).

This topic describes the role of a business service in this mechanism. Suppose an incoming event arrives in a production along with a deferred response token, and suppose the arrival point for this event is a business service. This business service then calls **SendDeferredResponse()** to create a response and direct it to the caller that originated the request. The **SendDeferredResponse()** call looks like this:

ObjectScript

```
Set sc = ..SendDeferredResponse(token, pResponseBody)
```

Where:

- *token*—A string that identifies the deferred response so that the caller can match it to the original request. The business service obtains the token string through some mechanism unique to the production.

For example, if the external destination is email, when sending a request for which it is willing to receive a deferred response, a business operation can include the token string in the subject line of the outgoing email. The entity receiving this email can extract this token from the request subject line and use it in the response subject line. This preserves the token so that the business service receiving the response email can use it in a subsequent call to **SendDeferredResponse()**.

- *pResponseBody*—A response message. This object receives the data returned by the response. See [Defining Messages](#).

This method returns a status, an instance of %Status.

6.6 Sending Multiple Request Messages

Your implementation of **OnProcessInput()** can send multiple request messages (for example, to different targets within the production). By default, all messages are sent within the same session. If you want each message to be sent within its own session, set the %SessionId property of the business service to null between sending messages. For example:

```
set ..%SessionId=""
set sc=..SendRequestSync("Test.FileOperation1",pOutput)
set ..%SessionId=""
set sc=..SendRequestSync("Test.FileOperation2",pOutput)
set ..%SessionId=""
set sc=..SendRequestSync("Test.FileOperation3",pOutput)
```

6.7 Processing Only One Event Per Call Interval

If you want the business service to process only one event per call interval, set the %WaitForNextCallInterval property to 1 (true) in your implementation of **OnProcessInput()**:

ObjectScript

```
set ..%WaitForNextCallInterval=1
```

This restricts the business service to processing only one input event per CallInterval, even when multiple input events exist.

This information applies to business services that use an adapter that have a property named CallInterval and that use that property as a polling interval.

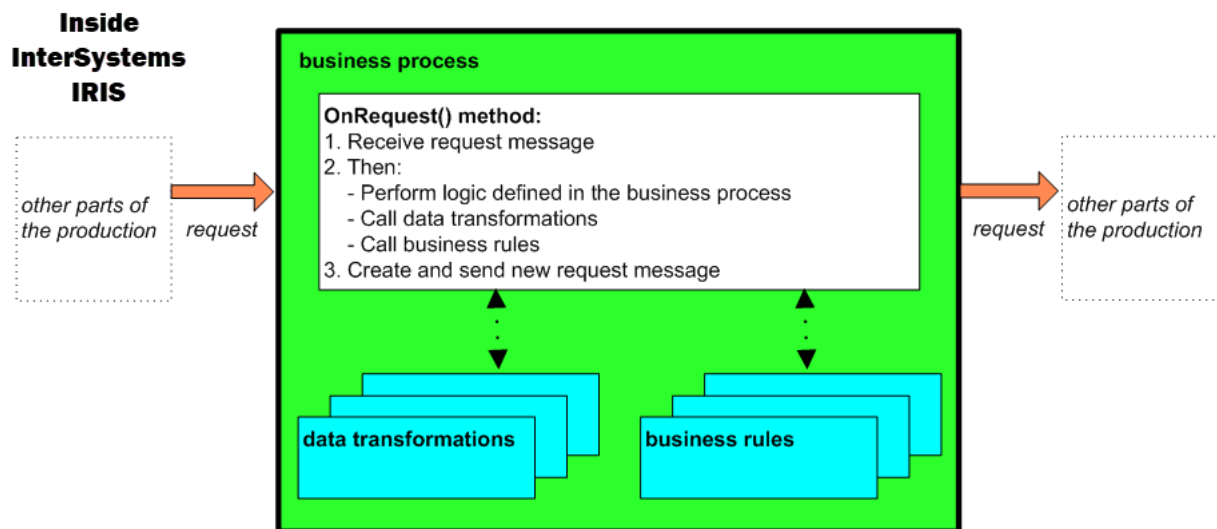
7

Defining Business Processes

Business processes are responsible for the higher level processing within a production. This page introduces them and discusses how to design and develop business process classes.

7.1 Introduction to Defining Business Processes

By convention, business processes contain most of the logic of the production. They can contain their own logic and they can call business rules and data transformations, each of which also contains specialized logic. The following figure illustrates this:



Note that this figure shows only the request messages.

There are many possible uses for business processes. In some cases, a business process coordinates a series of actions in one or more external applications. It contains the logic to determine the processing and it calls business operations or other business processes as needed. Business processes can also include human interaction; for details, see [Developing Workflows](#).

InterSystems IRIS® data platform provides the following general types of business process:

- *BPL processes*, which are based on the class `Ens.BusinessProcessBPL`.

Only BPL processes support the [business process execution context](#) and a graphical display of logic.

- *Routing processes*, which are based on the class `EnsLib.MsgRouter.RoutingEngine` or `EnsLib.MsgRouter.VDocRoutingEngine`.

InterSystems IRIS provides a set of classes to route specific kinds of messages. To use these subclasses, no coding is generally necessary. See [Types of Business Processes](#).

- *Custom business processes*, which are based on the class `Ens.BusinessProcess`.

A production can include any mix of these business processes.

Note that `Ens.BusinessProcessBPL`, `EnsLib.MsgRouter.RoutingEngine`, and `EnsLib.MsgRouter.VDocRoutingEngine` are all based on `Ens.BusinessProcess`.

7.2 Comparison of Business Logic Tools

You will probably develop business processes in conjunction with the data transformations and business rules that they use. Data transformations and business rules are intended to contain specific kinds of logic:

- Data transformations alter the message
- Business rules return values or specify where to send messages (or potentially both)

There is, however, overlap among the options available in business processes, data transformations, and business rules. To assist you in determining how to create these items, the following table compares them. It discusses BPL (the most common business process), DTL (the most common data transformations), and business rules.

Option	Supported in BPL?	Supported in DTL?	Supported in business rules?
Retrieving information about the business process	Yes (business execution context variables)	No	No
Assigning a value	Yes (<assign>)	Yes (assign action)	Yes (assign action)
Calling a data transformation	Yes (<transform>)	Yes (subtransform action)	Yes (send action)
Calling a business rule	Yes (<call>)	No	Yes (delegate action)
Calling custom code	Yes (<code>)	Yes (code action)	No
Invoking SQL	Yes (<sql>)	Yes (sql action)	No
Conditional logic	Yes (<if> , <switch> , <branch>)	Yes (if action)	No
Looping	Yes (<foreach> , <while> , <until>)	Yes (for each action)	No
Sending an alert	Yes (<alert>)	No	No
Including trace elements	Yes (<trace>)	Yes (trace action)	Yes (trace action)
Sending a request message to a business operation or process	Yes (<call>)	No	Yes (send action)

Option	Supported in BPL?	Supported in DTL?	Supported in business rules?
Waiting for a response from asynchronous requests	Yes (<sync>)	No	No
Deleting the message	No	No	Yes (delete action)
Performing error handling	Yes (<throw> , <catch> , and others)	No	No
Delaying execution for a specified duration or until a future time	Yes (<delay>)	No	No
Sending a primary response before execution is complete	Yes (<reply>)	No	No
Using XPATH and XSLT	Yes (<xpath> , <xslt>)	No	No
Storing a message temporarily to acknowledge a milestone	Yes (<milestone>)	No	No

For details on DTL transformations and business rules, see [Developing DTL Transformations](#) and [Developing Business Rules](#).

7.3 Key Principles for Business Processes

First, be sure to read [Programming in InterSystems IRIS](#).

When you develop business processes, consider the following key principles:

- Sometimes it is desirable to make the response object be a modified version of the incoming request object, and it may be useful to make modifications in stages. However, do not modify the incoming request object. Instead copy it to a context variable (or, for a custom business process, copy data to local variable). Then modify the copy.
- Be careful when sending messages synchronously (which you can do only within a custom business process or within `<code>` in BPL).

When a business process A calls business process B synchronously, process A does not continue until the response is received. If process A requires completion of calls to other processes (B) in order to complete itself, and if those processes share the pool of actor jobs, the actor pool can become deadlocked if there is no free actor job to process the called business process (B).

This happens because the calling business process cannot complete and free the actor job until the called business process returns, but the called business process cannot execute because there is no free actor job to execute it.

Also note that InterSystems IRIS cannot shut down during a true synchronous call.

It is better to use **SendRequestAsync()** and handle response messages in the **OnResponse()** method. If you need to call synchronously, you can avoid this problem by configuring the called business process (B) to use its own job pool.

- If a single-job business process makes a request and waits for the response, the process loses FIFO capability.

Because synchronous calls from BPL are implemented asynchronously by the compiler, the business process will go to disk after issuing the call. The actor can then continue to dequeue fresh business process or responses to other business processes.

FIFO processing is only guaranteed for a single-job business process if it makes no call requests that need a response. If a business process must make a call and still maintain FIFO, it must use a `<code>` activity that invokes **SendRequestSync()**. However, in that case the preceding bullet item applies.

- BPL provides a useful advantage: It handles every aspect of synchronous and asynchronous messaging in a streamlined and elegant way that prevents common problems with contention and deadlocks. Even when a call is marked synchronous, if any wait time is required, the production framework quietly frees the job that was executing the call on behalf of the BPL business process so that it can do other work while BPL waits for the synchronous response. Later on, the framework silently coordinates receipt of the synchronous response and wakes up the BPL business process to resume its work.

If you use custom code, it is very easy to (accidentally) design a production prone to deadlock. All you need to do is to create a sequence of business processes that send synchronous requests and use a limited actor pool. If the production then receives the same number of messages as actors, and a synchronous send occurs simultaneously for all the messages, the production ends up in a deadlock where all the actor processes are in the state of waiting for another process to release messages from queues. The most dangerous aspect of this issue is that testing does not necessarily produce the conditions that cause the deadlock. It is quite possible that the deadlock will first be encountered after deployment. At this point it will not be clear why the production has seized up, and the costs of this unforeseen problem may be high.

7.4 Defining BPL Business Processes

A BPL business process is a class based on `Ens.BusinessProcessBPL`. In this case, you can create and edit the process visually within either the Management Portal or an IDE. For information, see [Developing BPL Processes](#).

7.5 Defining Custom Business Processes

To create a custom business process class, define a class as follows:

- The class must extend `Ens.BusinessProcess` (or a subclass).
- The class must implement the `OnRequest()` and `OnResponse()` methods, as described in next two sections.
- The class can add or remove settings. See [Adding and Removing Settings](#).
- The class can implement any or all of the startup and teardown methods. See [Overriding Start and Stop Behavior](#).
- The class can contain methods to accomplish work internal to itself.

7.5.1 Implementing the `OnRequest()` Method

A custom business process class must implement the `OnRequest()` method. A production calls this method whenever an initial request for a specific business process arrives on the appropriate queue and is assigned a job in which to execute.

This method has the following signature:

Class Member

```
method OnRequest(request As %Library.Persistent, Output response As %Library.Persistent) as %Status {}
```

Where:

- *request*—The incoming request object.
- *response*—The response returned by this business process.

Note that if the business process received a message via **SendRequestAsync()**, no response is expected. In such a case, if the business process returns a response, that response is saved without a message header. If that is not what you want, the business process can programmatically check to see if a response is expected and create the response object only if needed. To see if a response is expected, use the `ReturnQueueName` property or the **needsReply()** method of the `..%PrimaryRequestHeader`. For example:

ObjectScript

```
if ..%PrimaryRequestHeader.ReturnQueueName='' {
    //create the response object
}
```

Or:

ObjectScript

```
if ..%PrimaryRequestHeader.needsReply() {
    //create the response object
}
```

7.5.1.1 Example

The following is an example of an **OnRequest()** method:

Class Member

```
Method OnRequest(request As Demo.Loan.Msg.Application, Output response As Demo.Loan.Msg.Approval)
    As %Status
{
    Set tSC=$$OK
    $$$TRACE("received application for "_request.Name)
    #;
    If $zcrcl(request.Name,2)#5=0 {
        Set tRequest = ##class(Demo.Loan.Msg.PrimeRateRequest).%New()
        Set tSC = ..SendRequestAsync("Demo.Loan.WebOperations",tRequest,1,"PrimeRate")
        If $$$ISOK(tSC){
            Set tRequest = ##class(Demo.Loan.Msg.CreditRatingRequest).%New()
            Set tRequest.SSN = request.SSN
            Set tSC = ..SendRequestAsync("Demo.Loan.WebOperations",tRequest,1,"CreditRating")
            If $$$ISOK(tSC){
                Set tSC = ..SetTimer("PT15S")
            }
        }
    } Else {
        Set response = ##class(Demo.Loan.Msg.Approval).%New()
        Set response.BankName = "BankUS"
        Set response.IsApproved = 0
        $$$TRACE("application is denied because of bank holiday")
    }
    Return tSC
}
```

7.5.2 Implementing the OnResponse() Method

A custom business process class must implement the `OnResponse()` method. A production calls this method whenever a response for a specific business process arrives on the appropriate queue and is assigned a job in which to execute. Typically this is a response to an asynchronous request made by the business process.

This method has the following signature:

Class Member

```
method OnResponse(request As %Library.Persistent,  
                  ByRef response As %Library.Persistent,  
                  callrequest As %Library.Persistent,  
                  callresponse As %Library.Persistent,  
                  pCompletionKey As %String) as %Status {}
```

This method takes the following arguments:

- *request*—The initial request object sent to this business process.
- *response*—The response object that will eventually be returned by this business process.
- *callrequest*—The request object associated with the incoming response.
- *callresponse*—The incoming response object.
- *pCompletionKey*—The completion key value associated with the incoming response. This is set by the call to the **SendRequestAsync()** method that made the request.

7.5.2.1 Example

The following is an example of an **OnResponse()** method:

Class Member

```
/// Handle a 'Response'  
Method OnResponse(request As Ens.Request,  
                  ByRef response As Ens.Response,  
                  callrequest As Ens.Request,  
                  callresponse As Ens.Response,  
                  pCompletionKey As %String) As %Status  
{  
    Set tSC=$$$$OK  
    If pCompletionKey="PrimeRate" {  
        Set ..PrimeRate = callresponse.PrimeRate  
    } Elseif pCompletionKey="CreditRating" {  
        Set ..CreditRating = callresponse.CreditRating  
    }  
    Return tSC  
}
```

7.5.3 Methods to Use in OnRequest() and OnResponse()

When you implement **OnRequest()** and **OnResponse()**, you can use the following methods of the `Ens.BusinessProcess` class:

- **SendDeferredResponse()**
- **SendRequestSync()**
But see [Key Principles for Business Processes](#).
- **SendRequestAsync()**
- **SetTimer()**
- **IsComponent()**

For details on these methods, see `Ens.BusinessProcess`.

8

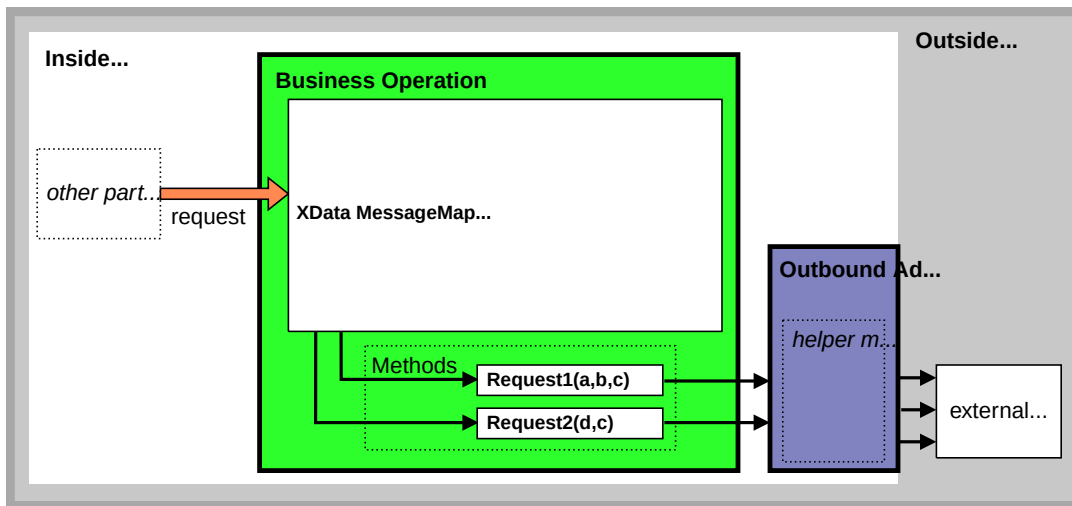
Defining Business Operations

This page describes how to define business operation classes.

Tip: InterSystems IRIS® data platform provides specialized business operation classes that use specific outbound adapters, and one of those might be suitable for your needs. If so, no programming would be needed. For a partial list, see [Connectivity Options](#).

8.1 Introduction to Defining Business Operations

A business operation is responsible for sending requests from InterSystems IRIS to an external application or system. The following figure shows how it works:



Note that this figure shows only the input flow of data, not the optional response.

A business operation is responsible for the following activities:

- Waiting for requests from business services or business processes.
- Dispatching, via a message map, the request to a specific method within the business operation. Each method within a business operation class represents a specific action within an external application.
- Transforming the request object into a form usable by the associated outbound adapter and asking the outbound adapter to send a request to the external application.

- Returning, if requested, a response object to the caller.

Each business operation contains a message map that specifies which external operation to perform, depending on the type of request message that it received. The message map contains one or more entries, each of which corresponds to one invocation of the associated outbound adapter.

8.2 Key Principles for Business Operations

First, be sure to read [Programming in InterSystems IRIS](#).

By convention, a business operation is an extremely specific operation that contains very little logic and does what is requested of it without calling further operations or branching in any way. When the design of the production demands logic, this is contained in a [business process](#).

Many productions provide a large set of extremely simple business operations. In these cases, business processes contain the logic that determines *when* each operation should be called.

Also see [Key Principles for Business Services](#).

8.3 Defining a Business Operation Class

To create a business operation class, define a class as follows:

- The class must extend `Ens.BusinessOperation` (or a subclass).
- The *ADAPTER* parameter should usually equal the name of the adapter class for this business service to use.

Or you can define a business operation with no associated outbound adapter class. In this case, the business operation itself must contain the logic needed to communicate with an external application.

- The *INVOCATION* parameter must specify the invocation style you want to use, which must be one of the following.
 - **Queue** means the message is created by the sending component's job and placed on a queue. The receiving component's job takes the message from the queue and processes it. This is the most common setting.
 - **InProc** means the message is created and processed by the sending component's job. (**InProc** stands for "in the same process.") Although the message trace display suggests that the message is sent to another job, the sending component's job just executes code in the receiving component's class. There is no message queue.
- The class should define a *message map* that includes at least one entry. A message map is an XData block entry that has the following structure:

```
XData MessageMap
{
  <MapItems>
    <MapItem MessageType="messageclass">
      <Method>methodname</Method>
    </MapItem>
    ...
  </MapItems>
}
```

See [Defining a Message Map](#).

- The class must define all the methods named in the message map. These methods are known as *message handlers*. Each message handler should have the following signature:

Class Member

```
Method Sample(pReq As RequestClass, Output pResp As ResponseClass) As %Status {}
```

Here *Sample* is the name of the method, *RequestClass* is the name of a request message class, and *ResponseClass* is the name of a response message class. In general, these methods will refer to properties and methods of the *Adapter* property of your business operation. For details, see [Defining Message Handler Methods](#).

- If *INVOCATION* is **Queue**, the class can define the optional *CONTEXT* parameter, which is a comma-separated list of variables to retain in the context between messages. If this parameter is set, all variables except those specified in this list will be killed before each message.

This parameter is ignored if *INVOCATION* is **InProc**.

- The class can add or remove settings. See [Adding and Removing Settings](#).
- The class can implement any or all of the startup and teardown methods. See [Overriding Start and Stop Behavior](#).

InterSystems IRIS is an integration platform potentially communicating with many other heterogeneous devices; therefore, it does not make property values dependent on server platform, time zone, time formatting, or other localization issues that may apply. Rather, InterSystems recommends you handle such cases in your production implementation. If your production requires different initial settings for property values, set the value in the **OnInit()** method of the business operation. See [Overriding Start and Stop Behavior](#).

- The class can contain methods to accomplish work internal to itself.

The following example shows the general structure that you need:

Class Definition

```
Class MyProduction.NewOperation Extends Ens.BusinessOperation
{
  Parameter ADAPTER = "MyProduction.MyOutboundAdapter";
  Parameter INVOCATION = "Queue";
  Method SampleCall(pRequest As Ens.Request, Output pResponse As Ens.Response) As %Status
  {
    Quit $$$ERROR($$$NotImplemented)
  }

  XData MessageMap
  {
    <MapItems>
      <MapItem MessageType="Ens.Request">
        <Method>SampleCall</Method>
      </MapItem>
    </MapItems>
  }
}
```

For examples of business operation classes, see the adapter guides.

8.4 Defining a Message Map

A message map is an XML document, contained within an XData MessageMap block in the business operation host class. For example:

Class Definition

```
Class MyProduction.Operation Extends Ens.BusinessOperation
{
  XData MessageMap
  {
    <MapItems>
      <MapItem MessageType="MyProduction.MyRequest">
        <Method>MethodA</Method>
      </MapItem>
      <MapItem MessageType="Ens.StringRequest">
        <Method>MethodB</Method>
      </MapItem>
    </MapItems>
  }
}
```

The operation of the message map is straightforward. When the business operation receives an incoming request, it searches, starting at the top of the message map, through each *MapItem* until it finds the first one whose *MessageType* attribute matches the type of the incoming message. It then invokes the operation method associated with this *MapItem*.

Some things to keep in mind about message maps:

- The message map is searched from top to bottom; once a match is found, no more searching is performed.
- If the incoming request object is a *subclass* of a given *MessageType* then it is considered a match. If you want to filter out subclasses, be sure to place them above any super classes within the message map.
- If the incoming request does not match any of the *MapItem* entries, then the **OnMessage** method is called.

8.5 Defining Message Handler Methods

When you create a business operation class, typically the biggest task is writing message handlers for use with this adapter, that is, methods that receive production messages and then invoke methods of the adapter in order to communicate with targets outside the production.

Each message handler method should have the following signature:

Class Member

```
Method Sample(pReq As RequestClass, Output pResp As ResponseClass) As %Status {}
```

Here *Sample* is the name of the method, *RequestClass* is the name of a request message class, and *ResponseClass* is the name of a response message class.

In general, the method should do some or all of the following:

1. Optionally set properties of the business operation class (at any appropriate time). See [Business Operation Properties](#).
2. Examine the input object.
3. Create an instance of the response class.
4. Call the applicable method or methods of the adapter. These methods are available via the Adapter property of your business operation. For example:

ObjectScript

```
Set tSc=..Adapter.SendMail(email,.pf)
```

This method is discussed after these steps.

Or, to send messages to a target within the production, see [Sending Messages to a Target within the Production](#).

5. Examine the response.
6. Use information in the response to create a response message (an instance of `Ens.Response` or a subclass), which the method returns as output.

For information on defining message classes, see [Defining Messages](#).

7. Make sure that you set the output argument (`pOutput`). Typically you set this equal to the response message. This step is required.
8. Return an appropriate status. This step is required.

8.6 Business Operation Properties

Within an operation method, the following properties of the business operation class are available:

Property	Description
<code>%ConfigName</code>	The configuration name for this business operation.
<code>%SessionId</code>	The session ID of the current message being processed.
<code>Adapter</code>	The associated outbound adapter for this business operation.
<code>DeferResponse</code>	To defer the response from this business operation for later delivery, set the <code>DeferResponse</code> property to the integer value 1 (true) and obtain a deferred response delivery token before exiting the business operation.
<code>FailureTimeout</code>	The length of time (in seconds) during which to continue retry attempts. After this number of seconds has elapsed, give up and return an error code. See <code>Retry</code> and <code>RetryInterval</code> .
<code>Retry</code>	Set this property to the integer value 1 (true) if you want to retry the current message. Typically, the retry feature is used when the external application is not responding and you wish to retry without generating an error. See <code>RetryInterval</code> and <code>FailureTimeout</code> .
<code>RetryInterval</code>	How frequently (in seconds) to retry access to the output system if this message is marked for retry. See <code>Retry</code> and <code>FailureTimeout</code> .
<code>SuspendMessage</code>	Set this property to the integer value 1 (true) if you want the business operation to mark its current in-progress message as having Suspended status. See the section Suspending Messages .

8.7 Calling Adapter Methods

Most commonly, a business operation does not contain the logic used to communicate with the external system. Rather, the business operation uses an outbound adapter that handles this logic. Once the business operation has been associated with an outbound adapter, it calls the adapter's methods to send and receive data. For details on invoking adapter methods, see [Accessing Properties and Methods from a Business Host](#).

8.8 Sending Requests to Targets within the Production

Although a business operation is primarily responsible for delivering a request to the specific external application, it can also send messages to other business operations or to business processes, as needed. To send messages to a target within the production, call **SendRequestSync()**, **SendRequestAsync()**, or **SendDeferredResponse()**.

For information on these methods, see [Sending Request Messages](#) in [Defining Business Services](#).

Ens.BusinessOperation defines an additional method that you can use: **DeferResponse()**.

8.8.1 The DeferResponse() Method

This method returns a %Status value indicating success or failure. It provides one by-reference argument, *token*, which returns the deferred response delivery token required for a later call to **SendDeferredResponse()**. For example:

ObjectScript

```
Set sc=..DeferResponse(.token)
// Send the token out somewhere...
Quit $$$0
```

For an overview of deferred sending, see [Using Deferred Sending](#) in [Programming in InterSystems IRIS](#).

8.9 Suspending Messages

If you want the business operation to mark its current in-progress message as having Suspended status, set the business operation property **SuspendMessage** to the integer value 1 (true). Typically a business operation will do this for messages that have been rejected by the external system for some reason.

InterSystems IRIS places a Suspended message on a special queue so that a system administrator can diagnose the problem, fix the problem, and then resend the message. The system administrator can perform a simple resend (to the original target) or can send it to a new destination. For information, see [Resending Messages](#).

The following sample method is from a business operation that sends a document to an external system. The method sets the **SuspendMessage** property to 1 if an error returns from the call to **Validate()** the document that is about to be sent:

Class Member

```
Method validateAndIndex(pDoc As MyX12.Document) As %Status
{
  If ""=..Validation||'$method($this,"OnValidate",pDoc,..Validation,.tSC) {
    Set tSC=##class(MyX12.Validator).Validate(pDoc,..Validation)
  }
  Set:'$D(tSC) tSC=$$0K
  If $$$ISERR(tSC) {
    Set ..SuspendMessage=1
    Do ..SendAlert(##class(Ens.AlertRequest).%New($LB(
      ..%ConfigName,"Suspended document "_pDoc.%Id()-
      " because it failed validation using spec '"_
      ..Validation_"' with error '"_
      $$$StatusDisplayString(tSC)))
    Quit tSC
  }
  If ""'=..SearchTableClass {
    TRY {
      Set tSCStore=$classmethod(..SearchTableClass,"IndexDoc",pDoc)
      If $$$ISERR(tSCStore)
        $$$LOGWARNING("Failed to create SearchTable entries")
    }
    CATCH errobj {
      $$$LOGWARNING("Failed to invoke SearchTable class")
    }
  }
}
```

```
    }  
  }  
  Quit $$$OK  
}
```


9

Defining an Alert Processor

System alerts and user-generated alerts provide a way to inform users of problems in the production. An alert processor is a business host that notifies applicable users via email or other mechanism, about a problem that must be corrected. In many cases, you can define an alert processor without creating custom code. See [Monitoring Alerts](#) for information on adding an alert processor to a production. This page describes how to create an alert processor with custom code.

9.1 Background Information

Your business hosts can send alerts; see [Generating Alerts](#) in [Programming in InterSystems IRIS](#). InterSystems IRIS® data platform also automatically sends alerts on specific occasions, depending on the values of settings in the production. If the production includes a business host named **Ens.Alert**, InterSystems IRIS automatically sends a specialized request message (Ens.AlertRequest) to that business host. This business host is the *alert processor* for the production; any production can contain no more than one of these.

The alert processor can then use the information in this message to determine who must be contacted. There are a couple of general scenarios:

- If you can handle all alerts via the same output mechanism, the alert processor can be a business operation that uses the applicable adapter. See [Using a Simple Email Alert Processor](#) and [Using a Simple Outbound Adapter Alert Processor](#).
- If you need to route the alert, but do not need to track alert resolution, see [Using a Routing Alert Processor](#).
- If you want to track the alert resolution process, see [Using Alert Management to Track Alert Resolution](#).

In all cases, InterSystems IRIS also writes the information to the InterSystems IRIS [Event Log](#), with the type Alert.

Note: **Ens.Alert** is the required name of the *business host* that serves as the alert processor. Do not confuse this with a class name. The alert processor can use any class name.

9.2 Using a Simple Email Alert Processor

If it is appropriate to send all alerts via email, use the class EnsLib.Email.AlertOperation for the Ens.Alert component. This specialized business operation does the following:

- The *ADAPTER* parameter is specified as EnsLib.Email.OutboundAdapter.

This adapter provides settings to specify email recipients, as well as information needed to use an SMTP email server.

It also provides helper methods to send email via the configured server.

- The **OnMessage()** method is implemented. This method expects `Ens.AlertRequest` as input.

This method does the following:

1. Reads the alert text from the `Ens.AlertRequest`.
2. Creates an email message (an instance of `%Net.MailMessage`) and writes the alert text into that.
3. Sends the email message to all configured recipients.

You might be able to use this class without modification. Or you could create and use a subclass of it.

9.3 Using a Simple Outbound Adapter Alert Processor

If you can handle all the alerts via the same output mechanism, but you cannot use `EnsLib.Email.AlertOperation`, then create a business operation named `Ens.Alert` as follows:

- Specify the *ADAPTER* parameter as the name of a suitable adapter class.
- Implement the **OnMessage()** method. The signature of this method should be as follows:

Class Member

```
Method OnMessage(pRequest As Ens.AlertRequest, Output pResponse As Ens.Response) As %Status {}
```

In your implementation, call methods of the adapter as needed.

See [Defining Business Operations](#) and see the adapter documentation.

You might want to define the class so that details such as email addresses and telephone numbers are configurable. See [Adding and Removing Settings](#).

9.4 Using a Routing Alert Processor

If you need to contact users via multiple output mechanisms, the alert processor should be a business process that determines how to route the `Ens.AlertRequest` messages. In this case, your production must contain an additional business operation for each output mechanism, and the alert processor forwards messages to those business operations.

9.4.1 Defining the Alert Processor as a Routing Process

To define the alert processor as a routing process, create a business process class that can receive `Ens.AlertRequest` messages.

The business process would examine the messages and forward them to different business operations, depending on the alert contents and any logic that you include.

Your logic may need to consider the following factors:

- Different requirements for various users
- Different requirements depending on the time of day
- The organization's problem-solving policies and procedures

You could use the class `EnsLib.MsgRouter.RoutingEngine` as the `Ens.Alert` routing process. This class provides the setting **Business Rule Name**. If you specify this setting as the name of a routing rule set, this business host uses the logic in that rule set to forward all the messages that it receives.

9.4.2 Defining the Business Operations

You can define each of the needed business operations as described in [Using a Simple Email Alert Processor](#) or [Using a Simple Outbound Adapter Alert Processor](#).

9.5 Adding Custom Code to Alert Management

Alert management allows you to assign alerts to users, track the status of alerts, and manage the progress of resolving alerts. For an overview of alert management, see [Configuring Alert Management](#), which describes how to configure alert management components and define rules and data transformations for alert management. This section describes how to add custom code to the alert management components.

The alert management framework has the following architecture:

- Single persistent object for a managed alert throughout its life cycle.
- Alert Manager, Notification Manager, and Alert Monitor have the same overall internal structure. When one of these components is invoked, it performs its function in three phases:
 1. First, the component executes the **OnProcess** method if it is implemented by a subclass. By implementing this method, you can include custom code in the component. If the **OnProcess** method sets a flag indicating that processing is complete, the component exits.
 2. Next, the component evaluates its rule or for the Notification Manager its data transformation, which sets parameters that control the component's actions.
 3. Finally, the component performs its action based on either the parameters set by the rule or the defaults set by the component's configuration.
- The Alert Notification operation is a simpler component that formats and forwards messages to its destinations.

9.5.1 Alert Manager

The Alert Manager has the class `Ens.Alerting.AlertManager` and must be named `Ens.Alert`. The Alert Manager receives alerts from all production components. The Alert Manager can promote an alert to a Managed Alert based on the conditions specified in a rule. The Alert Manager sends Managed Alerts to the Notification Manager.

The Alert Manager executes in three phases:

1. If the component's class overrides the **OnCreateManagedAlert()** method, execute the override. You can provide custom code to process the alert request and create the managed alert in this method. If you do not want the base Alert Manager code to evaluate the rule, create the managed alert, and send it to the Notification Manager, you should set the `tProcessingComplete` parameter to 1. In that case, the Alert Manager takes no further action.
2. Evaluate the `CreateManagedAlertRule` rule. This rule has access to `tAlertContext`. If it returns a true value (1), the Alert Manager creates the managed alert. If it returns false, the Alert Manager does not create the managed alert and the alert is only written to the log. The alert context provides access to:
 - Incoming alert
 - Alert groups configured for the component that originated the alert

- Business partner configured for the component that originated the alert
- Alert owner

The rule can suppress promoting the alert to a Managed Alert by returning 0 or can promote the alert to a Managed Alert by returning 1.

3. If the rule sets `tCreateAlert` to 1, the Alert Manager creates a Managed Alert, or, if there is no `CreateManagedAlertRule` rule defined, the Alert Manager takes the default action and creates a Managed Alert. The Alert Manager creates the Managed Alert by calling the **OnCreateManagedAlert()** method, which can be overridden by a class extending `Ens.Alerting.AlertManager`. The default implementation of **OnCreateManagedAlert()** sets the production name in the Managed Alert and sets the current owner to be unassigned with the value as an empty string. If the Alert Manager creates a Managed Alert, it sends it to the Notification Manager.

9.5.2 Notification Manager

The Notification Manager has the class `Ens.Alerting.NotificationManager` and is responsible for determining what groups to notify and which notification operations to use.

The Notification Manager executes in three phases:

1. If the component's class overrides the **OnProcessNotificationRequest()** method, execute the override. If the override sets the `pProcessingComplete` parameter to 1, the Notification Manager does not evaluate the transformation or apply the default action.
2. Executes the data transformation if one is configured. See [Adding the Notification Manager and Defining Its Data Transformation](#) for information about the data transformation.
3. If the transformation sets the `target.Notify` property to 1 or if there is no data transformation, the Notification Manager sends the Alert Notification to the component listed in each target and passes the list of addresses to the target.

The Notification Manager does not receive or send the Managed Alert object, but instead uses the Notification Request object, which contains a reference to the persistent Managed Alert object.

9.5.3 Alert Monitor

The Alert Monitor queries for all open Managed Alerts where the current time exceeds the `NextActionTime` value. It makes the following SQL query:

SQL

```
SELECT ID FROM Ens_Alerting.ManagedAlert WHERE IsOpen = 1 AND NextActionTime <= ?
```

where the current time returned by `$$$timeUTC` is specified as the parameter.

The Alert Monitor handles each returned Managed Alert message separately. For each Managed Alert, it processes it in three phases:

1. If the component's class overrides the **OnProcessOverdueAlert()** method, execute the override. You can provide custom code to process the alert. If you do not want the base Alert Monitor code to evaluate the rule, update the managed alert, and send it to the Notification Manager, you should set the `tProcessingComplete` parameter to 1. In that case, the Alert Monitor takes no further action.
2. Evaluate the `OverdueAlertRule` rule. This rule has access to `tOverdueContext`. The overdue context provides access to:
 - Incoming alert

- Current time
- NewNextActionTime
- NewEscalationLevel

The rule can suppress sending the reminder by returning 0, can set the next time that the managed alert will be found by the Alert Monitor by setting the NewNextActionTime or can escalate or de-escalate the Managed Alert by setting NewEscalationLevel.

You can override the context of the alert rule and how the Alert Monitor processes the results:

- You can add additional information to the alert rule context by overriding the **GetOnOverdueAlertContext()** method.
- You can override how the Alert Monitor handles the rule results by overriding the **OnProcessOverdueRuleResult()** method. Otherwise, execute this method in the base class. The **OnProcessOverdueRuleResult()** method is responsible for escalating the managed alert. The override has access to the managed alert, tOverdueContext, tSendNotification, and tNotificationType. Note that you should either duplicate the functionality of the base class implementation or call it by calling `##super()` .

3. If the rule returns 1, the Alert Monitor sends the managed alert to the Notification Manager.

9.5.4 Notification Operation

The Notification Operation sends notifications to groups of users. If you are sending notifications using more than one kind of mechanism, you can have a separate Notification Operation for each transmission method.

10

Defining Data Transformations

This page discusses data transformations.

10.1 Introduction to Defining Data Transformations

A *data transformation* creates a new message that is a transformation of another message. When you *transform* a message, your data transformation sends a new message body that is a transformation of the original. Some of the transformations that occur during this process can include:

- Copying values from properties on the source to properties on the target.
- Performing calculations using the values of properties on the source.
- Copying the results of calculations to properties on the target.
- Assigning literal values to properties on the target.
- Ignoring any properties on the source that are not relevant to the target.

10.2 Defining DTL Transformations

A *DTL transformation* is a class based on `Ens.DataTransformDTL`. In this case, you can create and edit the transformation visually in the DTL editor, which you can access in the Management Portal or in an IDE. The DTL editor is meant for use by nontechnical users. See [Developing DTL Transformations](#).

10.3 Defining Custom Transformations

A *custom transformation* is a subclass of `Ens.DataTransform` that specifies:

- The name of the input (source) message class
- The name of the output (target) message class
- A series of operations that assign values to the properties of the output object

Each assignment operation consists of a call to the `Ens.DataTransform` class method **Transform()**. The argument is a simple expression that is evaluated to provide the value for one of the properties in the output class. The expression can contain:

- Literal values
- Any property in the general-purpose execution context variable called *context*
- Properties on the source object
- Functions and operators from the expression language
- Calls to methods provided by InterSystems IRIS® data platform
- Calls to user-provided methods

11

Defining Business Metrics

An business metric measures or calculates one or more values, typically related to the performance of a production, for display in dashboards or in the Production Monitor. This page describes how to create and display business metrics.

Note: As an alternative, you can use third-party business activity monitoring products with InterSystems IRIS® data platform. These products can interoperate with InterSystems IRIS via any of its connectivity technologies, including web services, JDBC, ODBC, and more.

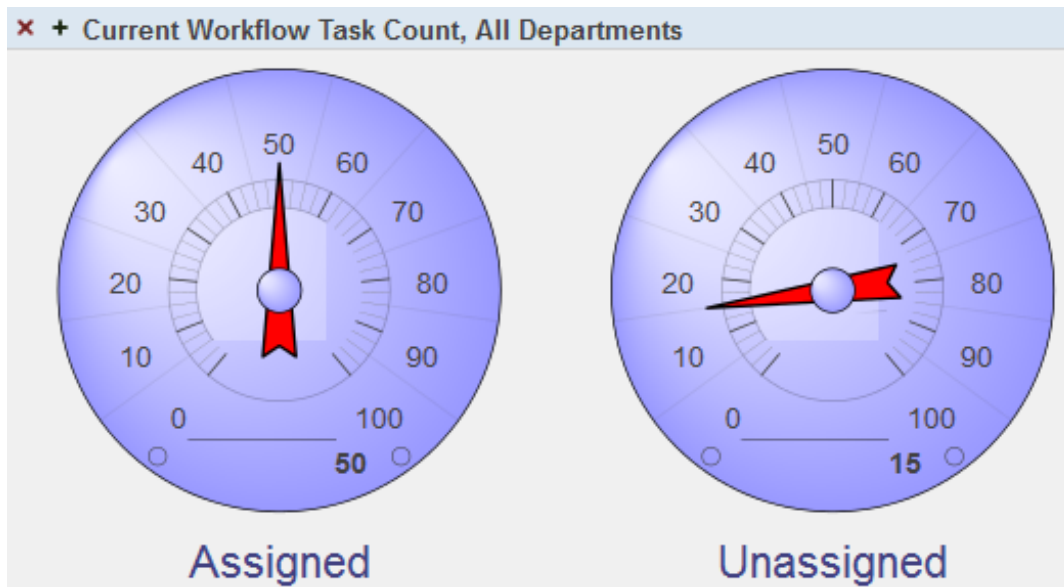
11.1 Introduction to InterSystems IRIS Business Metrics

A business metric is a specialized [business service class](#) that you include in a production. While the production is running, the business metric values are available for display; when it is not running, the values are null.

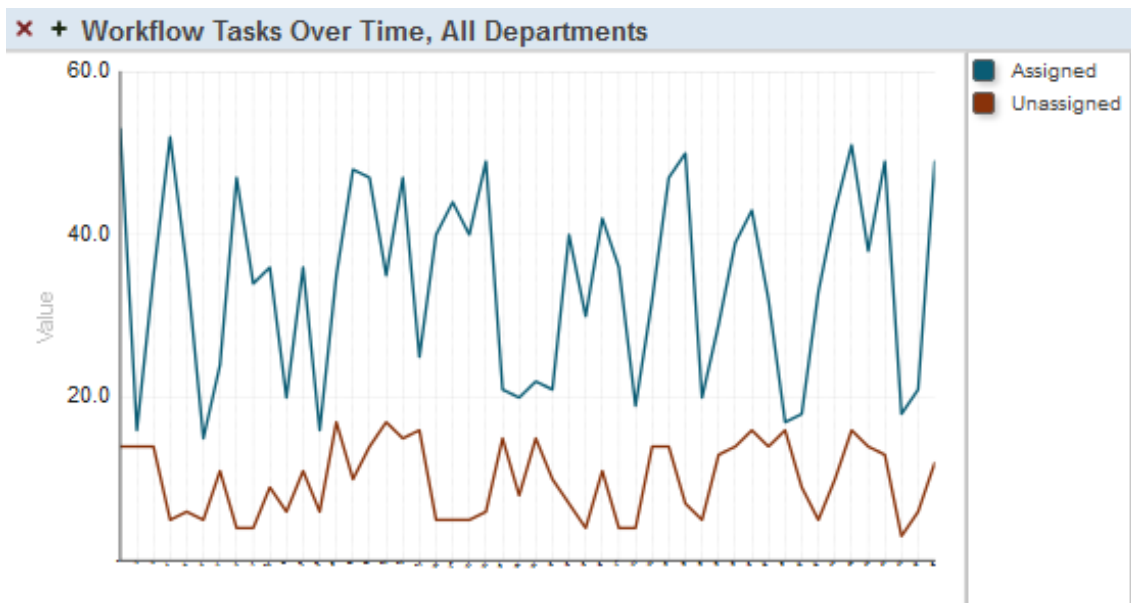
11.1.1 Business Metric Properties

The values in a business metric are called *properties*. There are two general kinds of business metric properties: simple properties and multidimensional properties with autohistory.

A *simple property* holds only one value at any time. The following example displays a business metric with two simple properties:



A *property with autohistory* holds multiple values, one value for each point in time, with the most recent value at the end. You can control the number of times when the value is recorded. The following example displays a business metric with two properties that have autohistory:



As a developer of business metric classes, you are free to provide values for metric properties in any manner you choose: such values can be based on data stored within production messages or business process instances, data maintained within InterSystems IRIS by business processes, or data obtained by sending requests to external applications or databases.

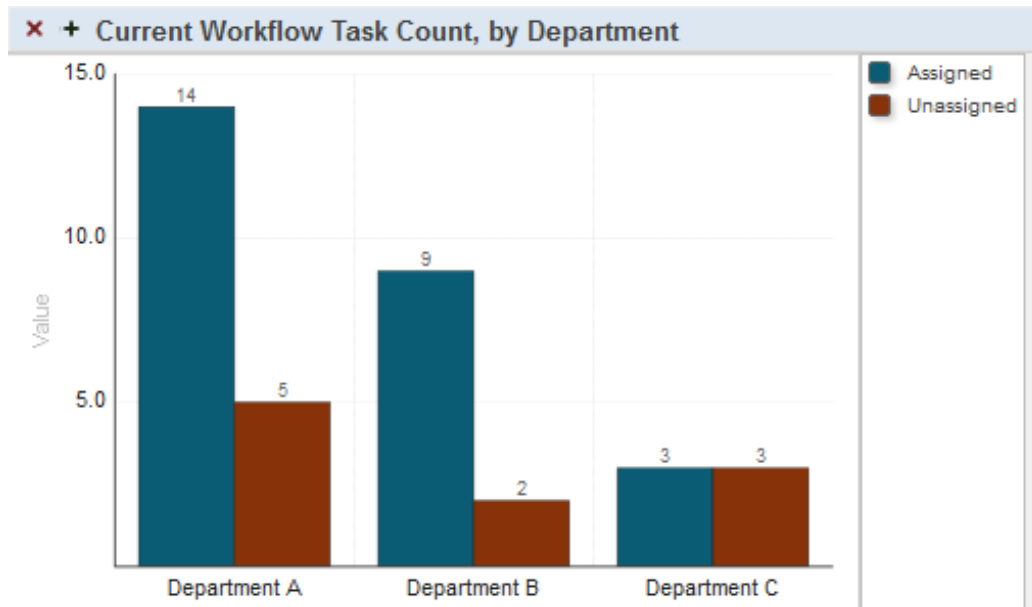
11.1.2 Single- and Multi-instance Business Metrics

Also, there are two general kinds of business metrics: single-instance and multi-instance business metrics.

A *single-instance business metric* holds a single set of metric values. The examples in the previous section showed single-instance business metrics.

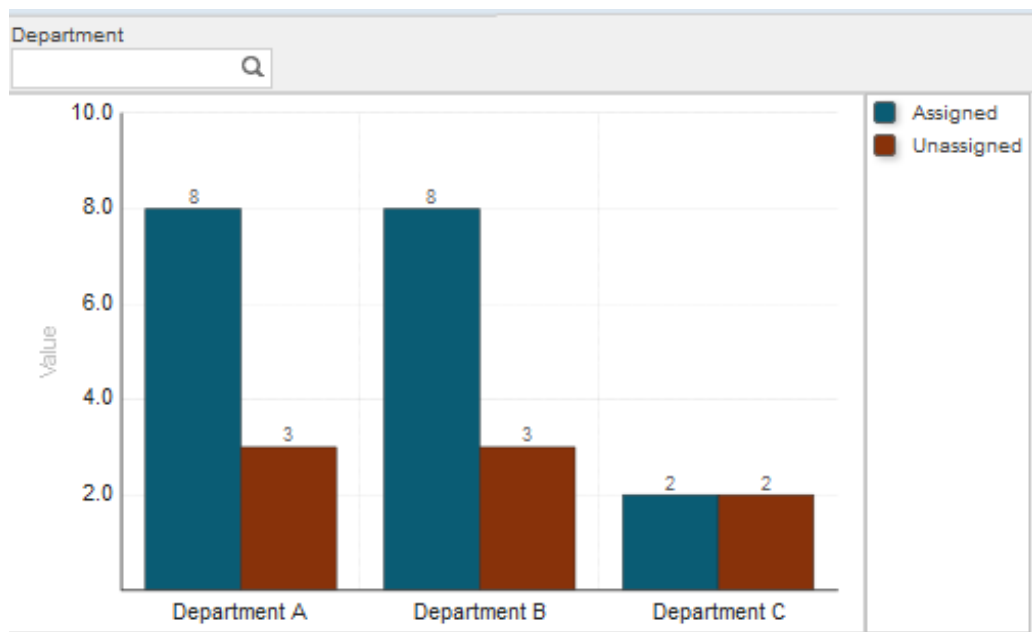
A *multi-instance business metric* holds multiple sets of metric values, one set for each instance defined by the metric. Multi-instance business metrics are useful when you have a number of similar items whose metrics you want to compare. Each

item is distinct, but has properties in common with the others. For example, each department might have a count of unassigned workflow tasks and a count of assigned workflow tasks. A business metric can have one instance for each department. The following shows an example multi-instance business metric that has two simple properties:

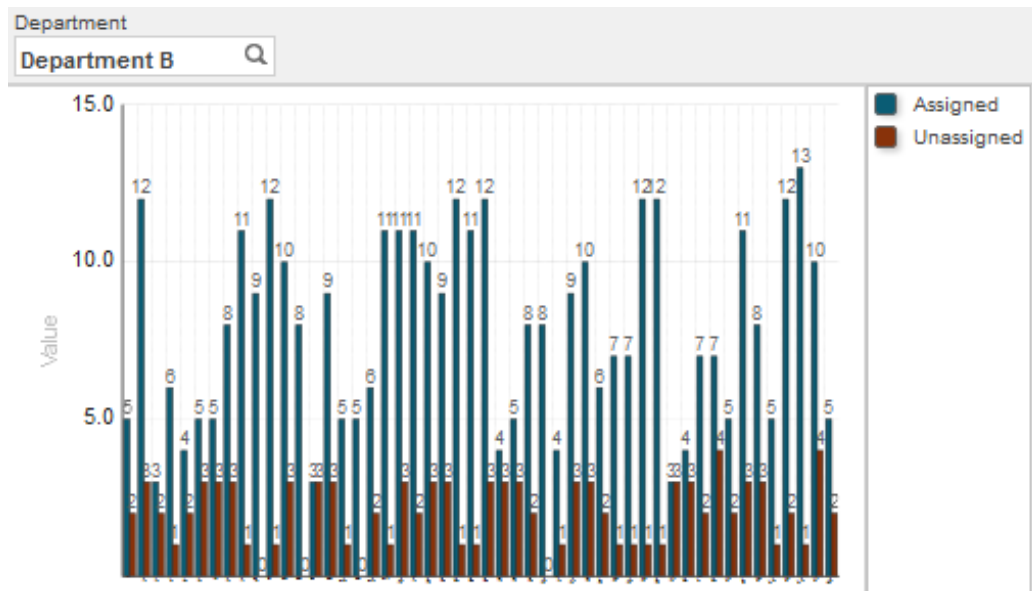


A multi-instance business metric can also have multidimensional properties with autohistory. In practice, however, it is not possible to simultaneously display the instances and the history. If you define such a business metric, then when you add it to the dashboard, the dashboard displays the current values for all instances by default. You can include a filter so that the user can select a single instance; in that case, the dashboard can display the history for that instance.

For example:



Then, when a user selects an instance:



11.1.3 Business Metrics as Business Services

All business metric classes are derived from the class `Ens.BusinessMetric`, which is itself derived from the `Ens.BusinessService` class so that it has the full functionality of a business service. For example, you can add a business metric to a production definition, you can assign a logical name to a business metric, you can schedule it for periodic execution (to recalculate its metric properties at a given interval), and you can invoke business operations and business processes as part of the metric value calculation.

11.2 Defining a Single-instance Business Metric

To define a single-instance business metric, define a class that meets the following requirements:

- It must be a subclass of `Ens.BusinessMetric`.
For this class, the *ADAPTER* parameter is `Ens.InboundAdapter`, which provides the **CallInterval** setting. This ensures that the business metric is invoked periodically.
- It must define one or more business metric properties. The details are different for [simple properties](#) and [properties with autohistory](#).
- It can optionally specify values for [property parameters](#), for example, to control the range of values.
- It must [assign values](#) to the business metric properties. To do so, it must implement the **OnCalculateMetrics()** method.

The following subsections provide the details.

Also see [Defining a Multi-instance Business Metric](#).

11.2.1 Defining Simple Business Metric Properties

To define a simple business metric property, add a property to the business metric class as follows:

Class Member

```
Property MetricProperty As Ens.DataType.Metric;
```

This property can hold either numeric or string values.

Where *MetricProperty* is the name of the business metric property. For example:

Class Member

```
/// This metric tracks A/R totals
Property AccountsReceivable As Ens.DataType.Metric;
```

This property can hold either numeric or string values.

11.2.2 Defining Business Metric Properties with Autohistory

To define a business metric property with autohistory, add a property to the business metric class as follows:

Class Member

```
Property MetricProperty As Ens.DataType.Metric (AUTOHISTORY=50) [MultiDimensional];
```

For the *AUTOHISTORY* parameter, you can use any positive integer. For example:

Class Member

```
/// Recent Sales History
Property SalesHistory As Ens.DataType.Metric (AUTOHISTORY = 50) [ MultiDimensional ];
```

Generally, the purpose of this kind of property is to collect values at intervals, over time, so that the resulting series of numbers can be plotted on a chart. For this reason, the assigned values are typically numeric.

The rate of collection is controlled by the **Call Interval** setting of the configured business metric.

11.2.3 Specifying Other Parameters for Metric Properties

You can specify additional property parameters for the business metric properties. These parameters might include default values for the upper and lower limits that control the appearance of any meter that displays that metric. For example:

Class Member

```
/// Total Sales for the current day.
Property TotalSales As Ens.DataType.Metric (RANGELOWER = 0, RANGEUPPER = 50, UNITS = "$US");
```

The following table lists the property parameters that you can use with *Ens.DataType.Metric* (in addition to *AUTOHISTORY*, which is discussed in the previous section). These parameters apply to metric properties that hold numeric values; none of them apply when the metric property holds a string value.

Parameter	Description
<i>LINK</i>	Optional. A URL specifying a browser page related to this metric. If a dashboard user right-clicks on the associated meter and selects the Drill Down option, the browser will show this page.
<i>RANGELOWER</i>	Optional. A numeric value specifying the expected lower range for values of this metric. This value provides a default for the low range of any meters connected to this metric.
<i>RANGEUPPER</i>	Optional. A numeric value specifying the expected upper range for values of this metric. This value provides a default for the upper range of any meters connected to this metric.

Parameter	Description
<i>THRESHOLDLOWER</i>	Optional. A numeric value specifying the expected lower threshold value for this metric. This value provides a default for the lower threshold of any meters connected to this metric.
<i>THRESHOLDUPPER</i>	Optional. A numeric value specifying the expected upper threshold value for this metric. This value provides a default for the upper threshold of any meters connected to this metric.
<i>UNITS</i>	Optional. A user-defined string enclosed in double quotes that specifies the units for this metric. Examples include "\$US" or "Liters". This string appears in the bottom half of the dashboard display when the viewer clicks on the corresponding meter in the top half of the display.

11.2.4 Assigning Values to Business Metric Properties

This section describes how to assign values to business metric properties for single-instance business metrics; the details for multi-instance business metrics are discussed [later](#) in this page.

To assign values to business metric properties, implement the **OnCalculateMetrics()** instance method of the business metric class. The purpose of **OnCalculateMetrics()** is to calculate, look up, or otherwise set a value for any metric properties in the class.

The following example defines two metric properties, Counter and SalesHistory. In this example, the **OnCalculateMetrics()** method simply increments the single-valued Counter property and updates the SalesHistory property to random value:

Class Definition

```

/// Example Business Metric class
Class MyProduction.MyMetric Extends Ens.BusinessMetric
{
    /// Number of times these metrics have been calculated.
    Property Counter As Ens.DataType.Metric
        (RANGELOWER = 0, RANGEUPPER = 100000, UNITS = "Events");

    /// Total Sales for the current day.
    Property SalesHistory As Ens.DataType.Metric
        (RANGELOWER = 0, RANGEUPPER = 50, AUTOHISTORY = 50, UNITS = "$US")
        [ MultiDimensional ];

    /// Calculate and update the set of metrics for this class
    Method OnCalculateMetrics() As %Status
    {
        // set the values of our metrics
        Set ..Counter = ..Counter + 1
        Set ..SalesHistory = $GET(..SalesHistory) + $RANDOM(10) - 5

        Quit $$$OK
    }
}

```

Notes:

- Notice that you specify the value for a property in the same way for simple properties and for properties with autohistory. (If you are familiar with multidimensional properties, note that you specify the value only for the unsubscripted top node of the property, as shown here. Because of the *AUTOHISTORY* parameter, InterSystems IRIS generates code that automatically maintains the lower nodes in the array, which is an integer-subscripted array. For example, *SalesHistory(1)* is the oldest value in the *SalesHistory* property.)
- In this method, you can optionally invoke business operations and business processes. You can also invoke any APIs needed to compute the values.

- Never allow a property with autohistory to contain a null value. The null values are not displayed, causing bar charts and line charts to contain the wrong number of bars or entries. This can result in a mismatch between the axis labels and the items that they label. To prevent such problems, replace any null values with zero.
- Before the **OnCalculateMetrics()** method is called, all metric properties are set to their last calculated value (if any). After the **OnCalculateMetrics()** is called, the values of all the metric properties are stored in the business metric cache for subsequent use by dashboards or other interested parties (if any).

11.3 Defining a Multi-instance Business Metric

To define a multi-instance business metric:

- Follow the instructions in the [previous section](#), except that the details are slightly different for [implementing **OnCalculateMetrics\(\)**](#).
- Define the instance names. To do so, you can:
 - [Define a static set of instance names](#). To do so, implement the **OnGetInstances()** method to assign a fixed list of names to an array. This approach is useful if the set of instances is static.
 - [Define the instance names dynamically](#). To do so, add a **MetricInstances()** query to get a list of names from a column in an SQL database. This approach is useful if you expect the number or names of the items to change over time.
 - Combine these approaches. Use the **MetricInstances()** query to get an initial list and then use **OnGetInstances()** to add or replace names. (The business metric instance calls **MetricInstances()** first and then calls **OnGetInstances()**.)
- In your implementation of **OnCalculateMetrics()**, check the value of the %Instance property and [assign values](#) to the business metric properties as appropriate for that instance.
- Keep the following principles in mind:
 - The instance names are strings.
 - The instance names must be unique.
 - The instance names may be displayed to users on a dashboard, so use names that are concise, informative, and appropriate.
 - Try to keep the number of instances reasonable. Thousands of instances could be expensive to compute and difficult for users to understand.

The following subsections provide the details.

11.3.1 Defining a Static Set of Instance Names

To define a static set of instance names, override the **OnGetInstances()** method. This method is passed an array by reference. The **OnGetInstances()** method must fill this array with names using an ordinal number as a subscript, starting with 1. A simple example follows:

Class Member

```
/// Return an array of metric instances
ClassMethod OnGetInstances(ByRef pInstSet As %String) As %Status
{
    Set pInstSet(1) = "Apple"
    Set pInstSet(2) = "Banana"
    Set pInstSet(3) = "Cherry"
    Quit $$$OK
}
```

11.3.2 Defining the Instance Names Dynamically

To define the set of instance names dynamically, add a query to the business metric class as follows:

- The query must be named **MetricInstances()**.
- It must take no arguments.
- The query must contain an SQL SELECT statement.
- The first column returned by the query returns the instance names.

For example:

Class Member

```
/// Return current list of product names
Query MetricInstances() As %SQLQuery
{
    SELECT ProductName FROM MyApplication.Product
}
```

11.3.3 Assigning Values to Properties in a Multi-instance Metric

To assign values to properties in a multi-instance business metric, implement the **OnCalculateMetrics()** method. In your implementation, set the values as appropriate for each instance. To do so:

1. Check the value of the %Instance property. This property equals the name of one of the business metric instances.
(InterSystems IRIS automatically processes one instance at a time, sequentially. For each instance, InterSystems IRIS executes the **OnCalculateMetrics()** instance method.)
2. Set the values of the business metric properties as needed for that instance.

The following shows an example:

Class Member

```
Method OnCalculateMetrics() As %Status
{
    // get product name
    Set product = ..%Instance

    // find recent sales using SQL
    &SQL(SELECT SUM(Sales) INTO :sales
        FROM MyApplication.Product
        WHERE ProductName = :product)

    // update sales metric
    Set ..Sales = sales
    Quit $$$OK
}
```

This example uses the current instance name (%Instance) in an SQL query to retrieve the most recent sales data for that instance, then writes this data to the Sales property of the current instance of the class.

Note: You can also use the `%Instance` property elsewhere in the class when you want to substitute the current instance name.

11.4 Other Options in Business Metrics

This section describes other options within business metric classes.

11.4.1 Defining Actions for Use in Dashboards

A business metric class can define actions, which you can expose as user options in dashboards. An action can perform a combination of client-side activities (such as filtering and refreshing the dashboard) and server-side activities (such as invoking your own APIs). The action mechanism is quite general.

To define actions, implement the `%OnGetActionList()` and `%OnDashboardAction()` methods of the business metric class. For information on these methods, see [Defining Custom Actions](#).

11.4.2 Implementing OnInit()

You can also override the **OnInit()** callback of the business metric class, to initialize any properties, for example. If you do so, you must ensure that it explicitly calls the **OnInit()** method provided by its superclass `BusinessMetric`, as shown below. If not, the corresponding dashboard element does not display properly:

Class Member

```
Method OnInit() As %Status
{
    // . . .

    // invoke superclass implementation
    Quit ##super()
}
```

11.5 Adding Business Metrics to Dashboards

To add business metrics to dashboards, do the following:

1. Add them to the appropriate production, in the same way that you would add any other business service.
2. Configure the **Call Interval** setting as needed for each business metric.
3. Create dashboards and add business metrics to them. For information, see [Configuring Productions](#).
4. Optionally extend the [Production Monitor](#) page to show information from your business metrics. See the [next section](#).

11.6 Adding Business Metrics to the Production Monitor

In addition to displaying business metrics in dashboards, you can extend the [Production Monitor](#) page to show information from your business metric classes. To do so, set nodes in the `^Ens.Monitor.Settings` global in your namespace, as follows:

Node	Value
<code>^Ens.Monitor.Settings("MetricClasses", n, "Metric")</code>	Configuration name of the business metric, for the <i>n</i> th business metric. The Production Monitor page lists the business metrics in the order specified by <i>n</i> .
<code>^Ens.Monitor.Settings("MetricClasses", n, "Title")</code>	Display name for this business metric. The default is the configuration name of the business metric
<code>^Ens.Monitor.Settings("MetricClasses", n, "Instance")</code>	Instance name of this business metric. If the metric does not have instances, omit this. If the metric does have instances and you omit this, InterSystems IRIS uses the first instance, considering the order in which instances are defined.

For example, do the following in the Terminal:

ObjectScript

```
Set ^Ens.Monitor.Settings("MetricClasses",1,"Metric") = "MetricConfigName"
Set ^Ens.Monitor.Settings("MetricClasses",1,"Title") = "Title for Display"
Set ^Ens.Monitor.Settings("MetricClasses",1,"Instance") = "MetricInstanceName"
```

For each business metric that you add, the Production Monitor page indicates when it last updated the metric information, if there is any data for the given metric or instance, and whether or not the given metric is currently running.

For information on using the Production Monitor page, see [Monitoring a Production](#).

11.7 Setting and Getting Values Programmatically

In some cases, you might need programmatic access to metric properties. For example, you might want a business process to directly read or set a metric property. To do this, use the **GetMetric()** and **SetMetric()** class methods of `Ens.BusinessMetric`.

11.7.1 Using the GetMetric() Method

The **GetMetric()** class method reads the current value of a specified metric property from the business metric cache. Call this method as follows:

ObjectScript

```
Set value = ##class(Ens.BusinessMetric).GetMetric(metric,property)
```

Where *metric* is the name of the business metric (the configuration name, not the class name) and *property* is the name of the metric property. If **GetMetric()** is unable to read the specified value, it returns an empty string.

To read values from multidimensional metric properties, there is a third, optional parameter that specifies which subnode of the property to read. For example:

ObjectScript

```
Set value(1) = ##class(Ens.BusinessMetric).GetMetric(metric,property,1)
```

11.7.2 Using the SetMetric() Method

The **SetMetric()** class method sets the value of a specified metric property within the business metric cache. Call this method as follows:

ObjectScript

```
Set tSC = ##class(Ens.BusinessMetric).SetMetric(metric,property,value)
```

Where *metric* is the name of the business metric (the configuration name, not the class name), *property* is the name of the metric property, and *value* is the value to which the metric property should be set.

SetMetric() returns a %Status code indicating success or failure. It is possible that **SetMetric()** is unable to acquire a lock for the business metric and may fail for that reason.

To set all the values of a multidimensional metric property, create the array of values and then pass the array by reference. For example:

ObjectScript

```
For i=1:1:20 {
    Set data(i) = i*i
}
Set tSC = ##class(Ens.BusinessMetric).SetMetric("MyMetric","MyGraph",.data)
```

11.8 About the Business Metric Cache

So that InterSystems IRIS can retrieve metric values as efficiently as possible, it stores these values in a cache. This cache is the global `^IRIS.Temp.EnsMetrics`, which has the following structure:

```
^IRIS.Temp.EnsMetrics(Namespace,BusinessMetric,Instance,Property) = value
```

Where:

- *Namespace* is the namespace where the production with the metric is running.
- *BusinessMetric* is the production configuration name of the business metric.
- *Instance* is an instance number. Instances are numbered in the order in which they are defined.
- *Property* is the name of the business metric property.

12

Defining an Enterprise Message Bank

This page describes how to define an optional, specialized kind of production called the Enterprise Message Bank.

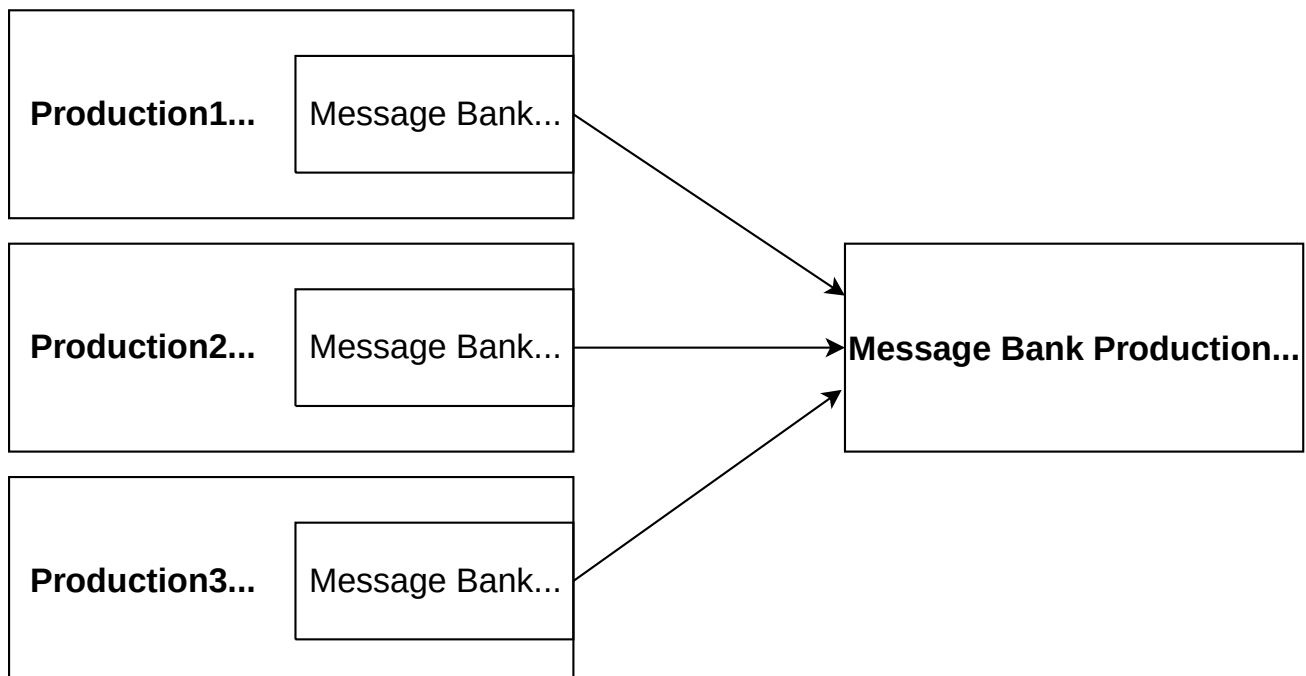
After you have defined the Enterprise Message Bank, see [Configuring the Enterprise Message Bank and Its Clients](#).

12.1 Enterprise Message Bank Overview

The Enterprise Message Bank is an optional remote archiving facility where you can collect messages, Event Log items, and search table entries from *multiple* client productions. It consists of the following components:

- The Message Bank server, which is a simple production consisting exclusively of a Message Bank service that receives submissions from any number of client productions.
- A client operation (the Message Bank operation) that you add to a production and configure with the address of a Message Bank server.

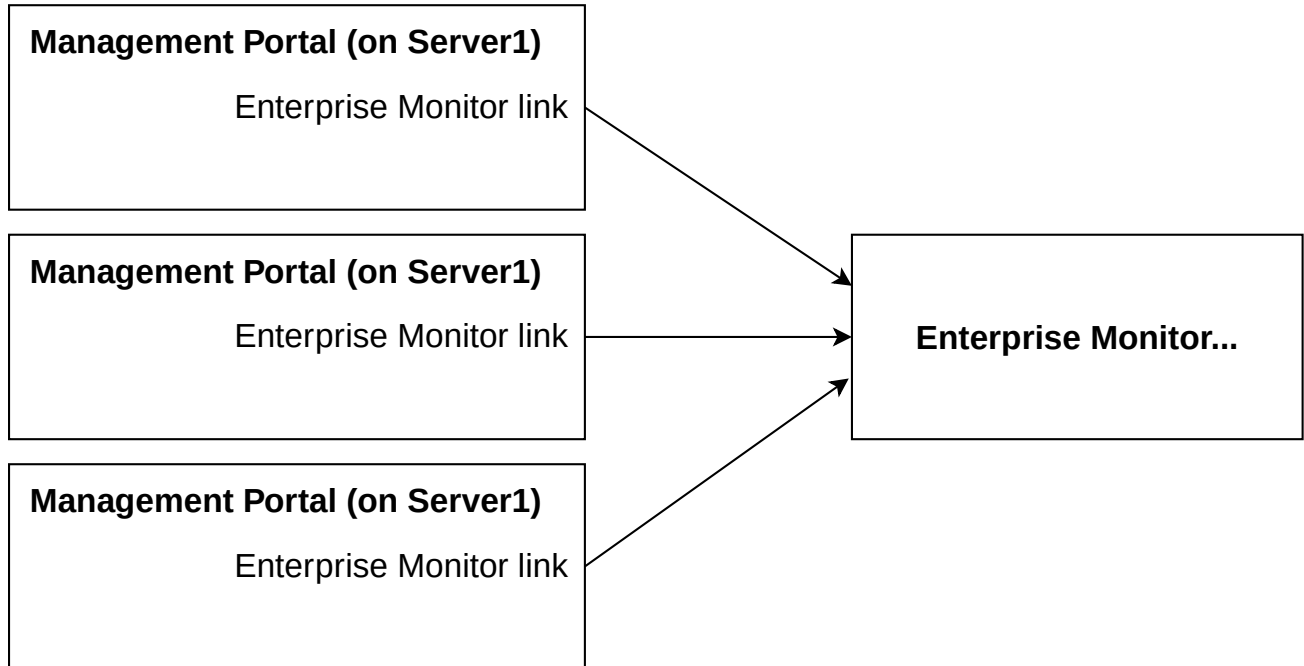
The following shows a conceptual example:



So that you can conveniently see the messages in the Message Bank, InterSystems IRIS® data platform provides the following additional options:

- For the Message Bank instance, the Management Portal automatically includes the Enterprise Monitor pages, where you can monitor the status of client productions, browse the Message Bank, and perform a search of messages from the monitored clients.
- You configure each client production to send messages and to include a convenient link to the Enterprise Monitor.

The following shows an example:



12.2 Defining the Message Bank Server

To define the Message Bank server, do the following on the server machine, in an interoperability-enabled namespace:

1. Create a subclass of the `Ens.Enterprise.MsgBank.Production` abstract class.
2. Copy its `ProductionDefinition` XData block into your class.
3. Compile your new class.

The following shows an example:

Class Definition

```

Class MyPkg.MyMessageBank Extends Ens.Enterprise.MsgBank.Production
{
  XData ProductionDefinition
  {
    <Production Name="Ens.Enterprise.MsgBank.Production" TestingEnabled="false" LogGeneralTraceEvents="false">
      <Description>Production for receiving and collating message bank submissions from one or more client
        interoperability-enabled namespaces and for maintaining a local repository of production status
        information about each client namespace, for display on the Enterprise Monitor page. Open the Monitor
        page on the same machine that is hosting this Production.</Description>
    </Production>
  }
}
  
```

```

<ActorPoolSize>0</ActorPoolSize>
<Setting Target="Production" Name="ShutdownTimeout">120</Setting>
<Setting Target="Production" Name="UpdateTimeout">10</Setting>
<Item Name="MonitorService" Category="" ClassName="Ens.Enterprise.MonitorService" PoolSize="1"
  Enabled="true" Foreground="false" InactivityTimeout="0" Comment="Populates global
  ^IRIS.Temp.Ens.EntMonitorStatus by polling namespaces from Systems List every CallInterval
seconds"
  LogTraceEvents="false" Schedule="">
  <Setting Target="Host" Name="AlertGracePeriod">0</Setting>
  <Setting Target="Host" Name="AlertOnError">0</Setting>
  <Setting Target="Host" Name="ArchiveIO">0</Setting>
  <Setting Target="Adapter" Name="CallInterval">10</Setting>
</Item>
<Item Name="MsgBankService" Category="" ClassName="Ens.Enterprise.MsgBank.TCPService" PoolSize="100"
  Enabled="true" Foreground="false" InactivityTimeout="20" Comment="" LogTraceEvents="true"
Schedule="">
  <Setting Target="Host" Name="AlertGracePeriod">0</Setting>
  <Setting Target="Host" Name="AlertOnError">0</Setting>
  <Setting Target="Host" Name="ArchiveIO">0</Setting>
  <Setting Target="Adapter" Name="Endian">Big</Setting>
  <Setting Target="Adapter" Name="UseFileStream">0</Setting>
  <Setting Target="Adapter" Name="JobPerConnection">1</Setting>
  <Setting Target="Adapter" Name="AllowedIPAddresses"></Setting>
  <Setting Target="Adapter" Name="QSize">100</Setting>
  <Setting Target="Adapter" Name="CallInterval">5</Setting>
  <Setting Target="Adapter" Name="Port">9192</Setting>
  <Setting Target="Adapter" Name="StayConnected">-1</Setting>
  <Setting Target="Adapter" Name="ReadTimeout">10</Setting>
  <Setting Target="Adapter" Name="SSLConfig"></Setting>
</Item>
</Production>
}
}

```

This production has the following services:

- Ens.Enterprise.MsgBank.TCPService—Accepts inbound data from the client productions. For details on configuring this service, see [Configuring the Message Bank Service on the Server](#).
- Ens.Enterprise.MonitorService—Collects status information from the client productions.

This production receives this data and maintains a local repository.

For testing purposes, you can put the Message Bank production on the same machine and instance as your regular productions, but it must be in a separate namespace from any production you plan to monitor.

12.3 Adding a Message Bank Helper Class

By default, the message bodies are not indexed and cannot be searched. You can add a helper class to implement the search capability in the Message Bank. To do so:

1. Create a subclass of Ens.Enterprise.MsgBank.BankHelperClass and implement its **OnBankMsg()** method. See the class reference for details on the method.

The **OnBankMsg()** method specifies any custom processing to perform when inbound messages include a message body.

In your implementation of this method, you should decide whether to handle messages in process (and possibly reduce the input capacity), or whether the method should asynchronously forward the messages to a secondary process to balance the work more effectively.

2. In the Message Bank production, select the Ens.Enterprise.MsgBank.TCPService business service and specify the **Bank Helper Class** setting. For the value, use the name of your helper class.

12.4 Notes about the Message Bank

Note the following important characteristics of the Message Bank:

- The Message Bank has no synchronization dependencies for message body classes with the contributing productions; therefore, it receives a serialized form of each message. For virtual document message bodies it reparses the serialized document into an object, and receives and stores the search table entries from the contributing production as well.
To enable searching by custom schema properties in the Message Bank, make sure to place the custom schema definition in the Message Bank production namespace.
- For uniqueness, the Message Bank production prepends the numeric identifier of client productions to the message IDs.
- The Message Bank page also serves as a portal for viewing the state of multiple client productions and for invoking a resend service on them. For these extra features to work, the Message Bank needs to know the web addresses of the client productions. For details, see [Configuring the Enterprise Message Bank](#).
- The Message Bank mechanisms do not delete the messages from the originating production; you should handle that function with a separate purge process.
- It is possible to replay messages from the Message Bank back into the originating production or another client production. See [Using the Enterprise Message Bank](#).

13

Less Common Tasks

This page discusses less common development tasks.

13.1 Defining Custom Utility Functions

InterSystems IRIS® data platform provides a set of utility functions that can be invoked from business rules and from DTL; these are described in [Utility Functions for Use in Productions](#). You can add your own functions, and the business rules engine and Business Rule Editor accommodate your extensions automatically.

To add a new utility function:

1. Create a new class that is a subclass of `Ens.Rule.FunctionSet`. The class can inherit from other classes as well, but only class methods defined in this class become utility functions.
2. For each function you wish to define, add a class method to your new function set class. There is no support for polymorphism, so to be precise, you must mark these class methods as `final`. You can view this in the existing `Ens.Util.FunctionSet` methods (`Ens.Util.FunctionSet` is a superclass of `Ens.Rule.FunctionSet`).
3. Compile the new class. The new functions are now available for use in rule expressions. To invoke these functions, use the `ClassMethod` name from your subclass. Unlike functions defined in `Ens.Rule.FunctionSet`, user-defined method names must be fully qualified with the class that they belong to. This happens automatically if you add them by selecting names from the wizards in the Management Portal.

As an example, the following function set class provides date and time functions for use in business rules. Its class methods **`DayOfWeek()`** and **`TimeInSeconds()`** invoke the ObjectScript functions `$ZDATE` and `$PIECE` to extract the desired date and time values from the ObjectScript special variable `$HOROLOG`:

Class Definition

```
/// Time functions to use in rule definitions.
Class Demo.MsgRouter.Functions Extends Ens.Rule.FunctionSet
{

  /// Returns the ordinal position of the day in the week,
  /// where 0 is Sunday, 1 is Monday, and so on.
  ClassMethod DayOfWeek() As %Integer [ CodeMode = expression, Final ]
  {
    $zd($H,10)
  }

  /// Returns the time as a number of seconds since midnight.
  ClassMethod TimeInSeconds() As %Integer [ CodeMode = expression, Final ]
  {
    $p($H, ",", ",2)
  }

}
```

For a full list of functions and special variables available in ObjectScript, see [ObjectScript Reference](#).

Once you have added a new function as described in this topic, the syntax for referring to it is slightly different than for built-in functions. Suppose you define this function in a class that inherits from `Ens.Rule.FunctionSet`.

Class Member

```
ClassMethod normalizaSexo(value as %String) as %String {}
```

After you compile the class, when you use one of the InterSystems IRIS Interoperability visual tools such as the Routing Rule Editor or Data Transformation Builder, you see your function name **normalizaSexo** included in the function selection box along with built-in functions like `Strip`, `In`, `Contains`, and so on.

Suppose you choose a built-in function from the function selection box and look at the generated code. You see that InterSystems IRIS has generated the function call using double-dot method call syntax (in DTL) or has simply referenced the function by name (in a business rule). (These syntax rules are explained in [Utility Functions for Use in Productions](#).)

The following example is an `<assign>` statement from DTL that references the built-in `Strip` function with double-dot syntax:

XML

```
<assign property='target.cod'
        value='..Strip(source.{PatientIDExternalID.ID}, "<>CW")'
        action='set' />
```

However, if you create your own, user-defined functions, the syntax for DTL is different. It is not enough simply to identify the function; you must also identify the full class name for the class that contains the class method for your function. Suppose your function **normalizaSexo** was defined in a class named `HP.Util.funciones`. In that case, after you chose a function from the function selection box and looked at the generated code, you would see something like the following example:

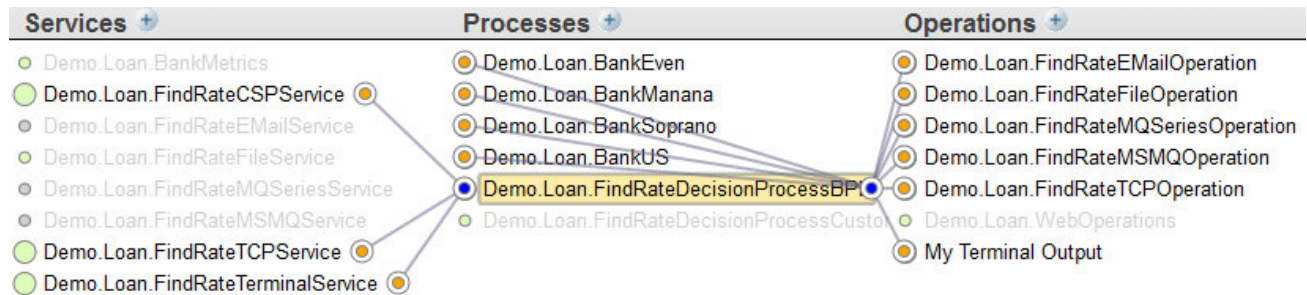
XML

```
<assign property='target.sexo'
        value='##class(HP.Util.funciones).normalizaSexo(source.{Sex})'
        action='set' />
```

You need to be aware of this syntax variation if you wish to type statements like this directly into your DTL code, rather than using the Data Transformation Builder to generate the code.

13.2 Rendering Connections When the Targets Are Dynamic

The Management Portal automatically displays the connections to and from a given business host, when a user selects that business host. For example:



To do this, InterSystems IRIS reads the configuration settings for the business host and uses them.

If, however, the business service host determines its targets dynamically, at runtime, InterSystems IRIS cannot automatically display such connections. In this case, to display such connections, implement the **OnGetConnections()** callback method. InterSystems IRIS automatically calls this method (which does nothing by default) when it renders the configuration diagram.

Note: If the business host uses `Ens.DataType.ConfigName` or one of its subclasses for connections, the connections will be rendered automatically and **OnGetConnections()** is not necessary.

OnGetConnections() has the following signature:

Class Member

```
ClassMethod OnGetConnections(Output pArray As %String, item As Ens.Config.Item) [ CodeMode = generator
{
}
```

Where the arguments are as follows:

- pArray*—A multidimensional array, whose subscripts are the configured names of items to which this business service sends messages. For example, to send messages to the business hosts named ABC and DEF, your code could set *pArray* as follows:

ObjectScript

```
set pArray("ABC")=""
set pArray("DEF")=""
```

- item*—The `Ens.Config.Item` object that represents this business service.

For examples of overridden **OnGetConnections()** methods, use an IDE to examine the built-in business services provided for use with electronic data interchange protocols such as X12. These are described in detail in [Routing X12 Documents in Productions](#) and other specific documentation.

13.3 Using Ens.Director to Start and Stop a Production

During development, you typically use the Management Portal to start and stop a production. For live, deployed production, InterSystems recommends that you use the auto-start option as described in [Configuring Productions](#).

Another option is to start or stop a production programmatically in the namespace where it is defined. To do so, invoke the following methods in the Ens.Director class:

StopProduction()

Stop the currently running production:

ObjectScript

```
Do ##class(Ens.Director).StopProduction()
```

StartProduction()

Start the specified production, as long as no other production is running:

ObjectScript

```
Do ##class(Ens.Director).StartProduction("myProduction")
```

This method starts a job that runs under the OS username used when the method was called; that user may or may not have appropriate permissions needed by the production. (In contrast, in the more common case, when the production calls this method, the job runs under the effective username of the instance, typically irisusr.)

RecoverProduction()

Clean up a Troubled instance of a running production so that you can run a new instance in the same namespace:

ObjectScript

```
Do ##class(Ens.Director).RecoverProduction()
```

It is not necessary to call **GetProductionStatus()** to see if the production terminated abnormally prior to calling **RecoverProduction()**. If the production is not Troubled, the method simply returns.

GetProductionStatus()

This method returns the production status via two output parameters, both of which are passed by reference. The first parameter returns the production name, but only when the status is Running, Suspended, or Troubled. The second parameter returns the production state, which is a numeric value equivalent to one of the following constants:

- \$\$\$eProductionStateRunning
- \$\$\$eProductionStateStopped
- \$\$\$eProductionStateSuspended
- \$\$\$eProductionStateTroubled

For example:

ObjectScript

```
Set tSC=##class(Ens.Director).GetProductionStatus(.tProductionName,.tState)
Quit:$$$ISERR(tSC)
If tState'=$$$eProductionStateRunning {
    $$$LOGINFO($$$Text("No Production is running. ")) Quit
}
```

You can use the production state macros such as \$\$\$eProductionStateRunning in code outside of the InterSystems IRIS production classes, for example in a general class or routine. To do this, you must add the following statement to the class:

```
#include Ensemble
```

It is not necessary to do this inside production classes, such as in business hosts.

Ens.Director provides many class methods, including many intended for use only by the InterSystems IRIS internal framework. InterSystems recommends that you use only the Ens.Director methods documented here, and only as documented.

Note: InterSystems recommends you *do not* use the ^%ZSTART routine to control production startup. The InterSystems IRIS startup mechanisms are much easier to use and are more closely tied to the production itself.

13.4 Using Ens.Director to Access Settings

The following Ens.Director class methods allow retrieval of production settings even when the production is not running:

GetAdapterSettings()

Returns an array containing the values of all adapter settings for the identified configuration item: a business service or business operation. The array is subscripted by setting name. You can use the InterSystems IRIS \$ORDER function to access the elements of the array. The first parameter for this method is a string that contains the production name and configuration item name separated by two vertical bars (| |). The return value is a status value. If the status value is not \$\$\$OK, the specified combination of production name (myProd) and configuration item name (myOp) could not be found.

ObjectScript

```
Set tSC=##class(Ens.Director).GetAdapterSettings("myProd|myOp",.tSettings)
```

GetAdapterSettingValue()

Returns the value of a named adapter setting for the identified configuration item: a business service or business operation. The first parameter is a string that contains the production name and configuration item name separated by two vertical bars (| |). The second parameter is the name of a configuration setting. The third output parameter returns a status value from the call. For example:

ObjectScript

```
Set val=##class(Ens.Director).GetAdapterSettingValue("myProd|myOp","QSize",.tSC)
```

If the returned status value is not \$\$\$OK, the specified combination of production name (myProd) and configuration item name (myOp) could not be found, or a setting of the specified name (QSize) was not found in the configuration for that specified production and configuration item.

GetCurrProductionSettings()

Returns an array containing the values of all production settings from the currently running production or the production most recently run. The array is subscripted by setting name. The return value for this method is a status value. If the status value is not \$\$\$OK, no current production could be identified.

ObjectScript

```
Set tSC=##class(Ens.Director).GetCurrProductionSettings(.tSettings)
```

GetCurrProductionSettingValue()

Returns the string value of a named production setting from the currently running production or the production most recently run. The second output parameter returns a status value from the call. If this status value is not \$\$\$OK, either a setting of the specified name was not found in the configuration for the current production, or no current production could be identified.

ObjectScript

```
Set myValue=##class(Ens.Director).GetCurrProductionSettingValue("mySet",.tSC)
```

GetHostSettings()

Returns an array containing the values of all settings for the identified configuration item: a business service, business process, or business operation. The array is subscripted by setting name. The first parameter for this method is a string that contains the production name and configuration item name separated by two vertical bars (|). The return value is a status value. If the status value is not \$\$\$OK, the specified combination of production name (myProd) and configuration item name (myOp) could not be found.

ObjectScript

```
Set tSC=##class(Ens.Director).GetHostSettings("myProd|myOp",.tSettings)
```

GetHostSettingValue()

Returns the value of a named setting for the identified configuration item: a business service, business process, or business operation. The first parameter is a string that contains the production name and configuration item name separated by two vertical bars (|). The second parameter is the name of a configuration setting. The third output parameter returns a status value from the call. For example:

ObjectScript

```
Set val=##class(Ens.Director).GetHostSettingValue("myProd|myOp","QSize",.tSC)
```

If the returned status value is not \$\$\$OK, the specified combination of production name (myProd) and configuration item name (myOp) could not be found, or a setting of the specified name (QSize) was not found in the configuration for that specified production and configuration item.

GetProductionSettings()

Returns an array containing the values of all production settings from the named production. The array is subscripted by setting name. The return value for this method is a status value. If the status value is not \$\$\$OK, the specified production could not be found.

ObjectScript

```
Set tSC=##class(Ens.Director).GetProductionSettings("myProd",.tSettings)
```

GetProductionSettingValue()

Returns the value of a named production setting from the named production. The third output parameter returns a status value from the call. If this status value is not \$\$\$OK, the specified production could not be found, or a setting of the specified name was not found in the configuration for the specified production.

ObjectScript

```
Set val=##class(Ens.Director).GetProductionSettingValue("prod","set",.tSC)
```

Ens.Director provides many class methods, including many intended for use only by the InterSystems IRIS internal framework. InterSystems recommends that you use only the Ens.Director methods documented here, and only as documented.

13.5 Invoking a Business Service Directly

There are times when you want to invoke a business service directly, from a job that has been created by some other mechanism, such as a language binding, CSP pages, SOAP, or a routine invoked from the operating system level. You can do so only if the value of the *ADAPTER* class parameter is null; this type of business service is called an *adapterless* business service.

For a business service to work, you must create an instance of the business service class. You *cannot* create this instance by calling the **%New()** method. Instead, you must use the method **CreateBusinessService()** of Ens.Director. For example:

ObjectScript

```
Set tSC = ##class(Ens.Director).CreateBusinessService("MyService",.tService)
```

A production does not allocate a job for this business service at production startup; it assumes a **Pool Size** setting of 0.

The **CreateBusinessService()** method does the following:

1. It makes sure that a production is running and that the production defines the given business service.
2. It makes sure that the given business service is currently enabled.
3. It resolves the configuration name of the business service and instantiates the correct business service object using the correct configuration values (a production may define many business services using the same business service class but with different names and settings).

If the **CreateBusinessService()** method succeeds, it returns, by reference, an instance of the business service class. You can then directly invoke its **ProcessInput()** method. This method is internal and cannot be seen in the class reference. Its signature is as follows:

```
Method ProcessInput(pInput As %RegisteredObject, Output pOutput As %RegisteredObject, ByRef pHint As %String) As %Status {
}
```

For example:

ObjectScript

```
If ($IsObject(tService)) {
    Set input = ##class(MyObject).%New()
    Set input.Value = 22
    Set tSC = tService.ProcessInput(input,.output)
}
```

Ens.Director provides many class methods, including many intended for use only by the InterSystems IRIS internal framework. InterSystems recommends that you use only the Ens.Director methods documented here, and only as documented.

13.6 Creating or Subclassing Inbound Adapters

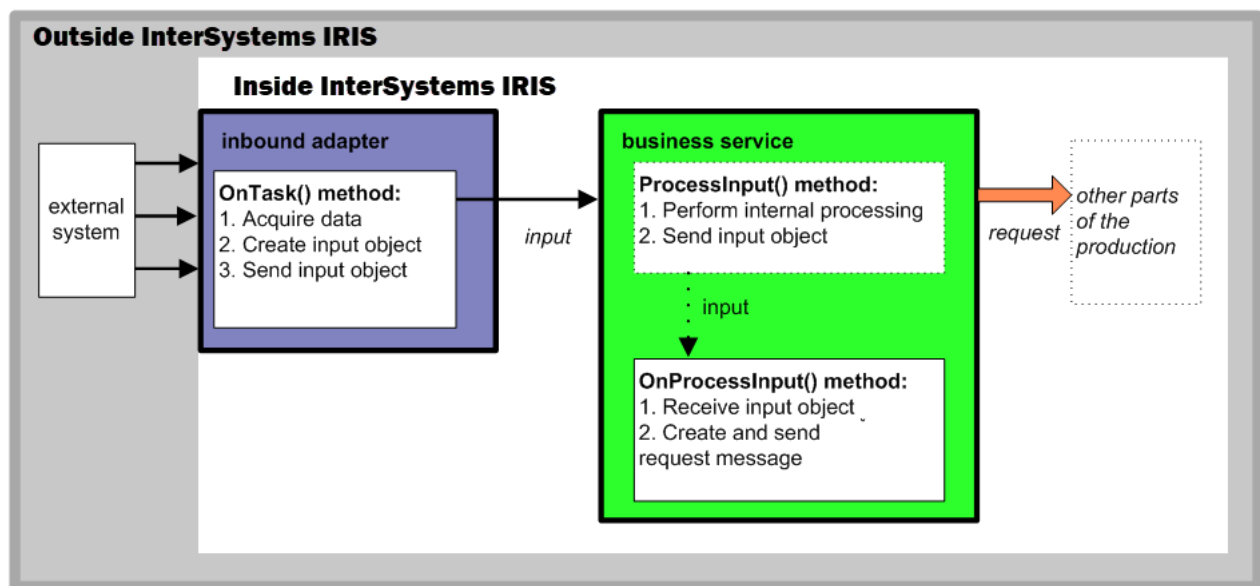
This section describes how to create or subclass an inbound adapter.

13.6.1 Introduction to Inbound Adapters

An inbound adapter is responsible for receiving and validating requests from external systems.

Inbound adapter classes work in conjunction with a business service classes. In general, the inbound adapter contains general-purpose, reusable code while the business service contains production-specific code (such as special validation logic).

Typically you implement inbound adapter classes using one of the InterSystems IRIS built-in adapter classes. The following figure shows how a production accepts incoming requests:



In general, when an external application makes a request for a certain action to be performed, the request comes into InterSystems IRIS via an inbound adapter, as shown in the previous figure. The requesting application is called a client application, because it has asked the production to do something. This application is a client of the production. The featured element at this step is the inbound adapter. This is a piece of code that adapts the client's native request format to a one that is understandable to the production. Each application that makes requests of a production must have its own inbound adapter. No change to the client application code is needed, because the adapter handles calls that are already native to the client application.

13.6.2 Defining an Inbound Adapter

To create an inbound adapter class, create a class as follows:

- The class must extend Ens.InboundAdapter (or a subclass).
- The class must implement the **OnTask()** method, as described in [Implementing the OnTask\(\) Method](#).
- The class can define settings. See [Adding and Removing Settings](#).

- The class can implement any or all of the startup and teardown methods. See [Overriding Start and Stop Behavior](#).
- The class can include production credentials. See [Including Credentials in an Adapter Class](#).
- The class can contain methods to accomplish work internal to itself.

The following shows an example:

Class Definition

```
Class MyProduction.InboundAdapter Extends Ens.InboundAdapter
{
  Parameter SETTINGS = "IPAddress,TimeOut";
  Property IPAddress As %String(MAXLEN=100);
  Property TimeOut As %Integer(MINVAL=0, MAXVAL=10);
  Property Counter As %Integer;
  Method OnTask() As %Status
  {
    #; First, receive a message (note, timeout is in ms)
    Set msg = ..ReceiveMessage(..CallInterval*1000,.tSC)

    If ($IsObject(msg)) {
      Set tSC=..BusinessHost.ProcessInput(msg)
    }
    Quit tSC
  }
}
```

13.6.3 Implementing the OnTask() Method

The **OnTask()** method is where the actual work of the inbound adapter takes place. This method is intended to do the following things:

1. Check for an incoming event. The inbound adapter can do this in many different ways: For example, it could wait for an incoming I/O event (such as reading from a TCP socket), or it could periodically poll for the existence of external data (such as a file).

Most prebuilt inbound adapters have a setting called `CallInterval` that controls the time interval between calls to **OnTask()**. You could use this approach as well.

2. Package the information from this event into an object of the type expected by the business service class.
3. Call the **ProcessInput()** method of the business service object. This method is internal and cannot be seen in the class reference. Its signature is as follows:

```
Method ProcessInput(pInput As %RegisteredObject, Output pOutput As %RegisteredObject, ByRef pHint
  As %String) As %Status {
}
```

4. If necessary, send an acknowledgment back to external system that the event was received.
5. If there is more input data, **OnTask()** can do either of the following:
 - Call **ProcessInput()** repeatedly until all the data is retrieved.
 - Call **ProcessInput()** only once per `CallInterval`, even if multiple input events exist.

When designing an inbound adapter, it is important to keep in mind that the **OnTask()** method must periodically return control to the business service; that is, the **OnTask()** method must not wait indefinitely for an incoming event. It should, instead, wait for some period of time (say 10 seconds) and return control to the business service. The reason for this is that

the business service object must periodically check for events from within InterSystems IRIS, such as notification that the production is being shut down.

Conversely, it is also important that the **OnTask()** method waits for events efficiently—sitting in a tight loop polling for events wastes CPU cycles and slows down an entire production. When an **OnTask()** method needs to wait, it should wait in such a way as to let its process go to sleep (such as waiting on an I/O event, or using the **Hang** command).

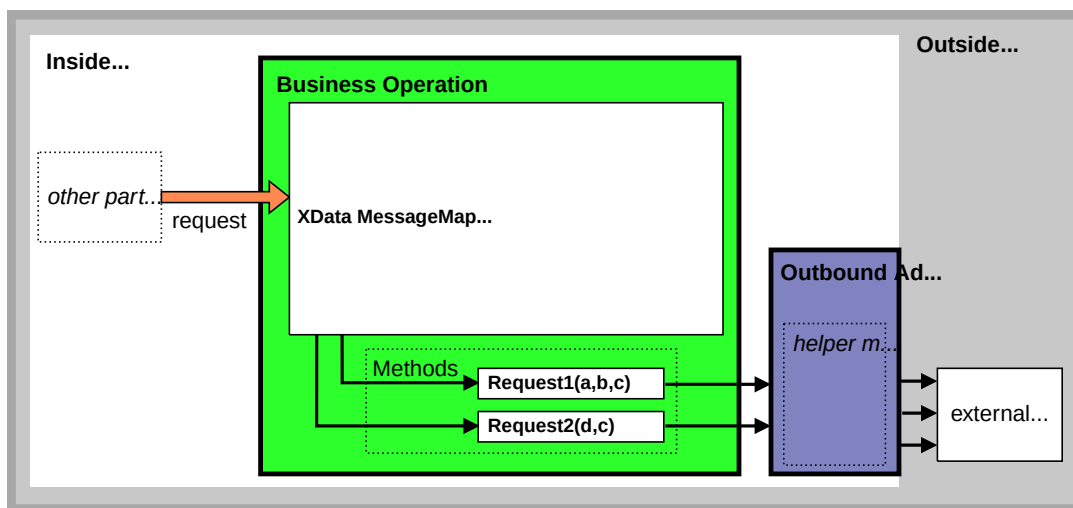
If your class is a subclass of a production adapter, then it probably implements the **OnTask()** method; therefore, your subclass might need to override a different method as specified by the inbound adapter class.

13.7 Creating or Subclassing Outbound Adapters

This section describes how to create or subclass an outbound adapter.

13.7.1 Introduction to Outbound Adapters

An outbound adapter is responsible for sending requests to external systems. The following figure shows how a production relays outgoing requests.



The outbound adapter is a piece of code that adapts the native programming interface of an external application or external database into a form that is understandable to a production. Each external application or database that serves a production by means of a business operation must have its own outbound adapter. However, not every method in the external application or database needs to be mapped to the outbound adapter; only those operations that the production requires. As with inbound adapters, no change to the external application itself is needed to create an outbound adapter. And, the adapter itself is conceptually simple: It relays requests, responses, and data between the production and a specific application or database outside the production.

Outbound adapter classes work in conjunction with a business operation classes. In general, the outbound adapter contains general-purpose, reusable code while the business operation will contain production-specific code (such as special processing logic). Typically you will implement your outbound adapter classes using one of the InterSystems IRIS built-in adapter classes.

13.7.2 Defining an Outbound Adapter

To create an outbound adapter class, create a class as follows:

- The class must extend `Ens.OutboundAdapter` (or a subclass).
- The class must define one or more methods for the corresponding business operation to invoke. Every outbound adapter is free to define its own API (set of methods) for use by the associated business operation classes.
- The class can define settings. See [Adding and Removing Settings](#).
- The class can implement any or all of the startup and teardown methods. See [Overriding Start and Stop Behavior](#).
- The class can include production credentials. See [Including Credentials in an Adapter Class](#).
- The class can contain methods to accomplish work internal to itself.

13.8 Including Credentials in an Adapter Class

To include production credentials in an adapter class, do the following in the class definition:

- Include a setting named `Credentials`.
- Define a method called **`CredentialsSet()`** that uses the value of the `Credentials` setting as a key to look up the username and password in the `Credentials` table. It then instantiates a credentials object that contains the username and password.

13.9 Overriding Production Credentials

While the production credentials system centralizes management and keeps login data out of source code, sometimes you need to write code that gets credentials from another source. For example, your code might retrieve a username and password from a web form or cookie, and then use them with the HTTP outbound adapter to connect to some other site.

The way to handle this is in your business service or business operation code, do both of the following, before calling any adapter methods:

- Provide code that instantiates a credentials object and assigns username and password values to it
- Do not subsequently set the adapter `Credentials` property or call the adapter **`CredentialsSet()`** method, or the values may be reset.

For example:

ObjectScript

```
If ..Adapter.Credentials="" {
    Set ..Adapter.%CredentialsObj=##class(Ens.Config.Credentials).%New()
}
Set ..Adapter.%CredentialsObj.Username = tUsername
Set ..Adapter.%CredentialsObj.Password = tPassword
```

Code such as this provides a credentials object that the `EnsLib.HTTP.OutboundAdapter` can use, but the values inside the object do not come from the `Credentials` table.

13.10 Overriding Start and Stop Behavior

InterSystems IRIS provides a set of callback methods that you can override in order to add custom processing at start and stop times during the life cycle of the [production](#), its [business hosts](#), or its [adapters](#). By default, these methods do nothing.

13.10.1 Callbacks in the Production Class

If you have code that must execute before a production starts up, but that requires the InterSystems IRIS production framework to be running before it can execute, you must override the **OnStart()** method in the production class. Place these code statements in **OnStart()** so that they execute in the proper sequence: that is, *after* InterSystems IRIS has started, but before the production begins accepting requests. The **OnStop()** method is also available to perform a set of tasks before the production finishes shutting down.

13.10.2 Callbacks in Business Host Classes

Each business host —business service, business process, or business operation—is a subclass of `Ens.Host`. In any of these classes you may override the **OnProductionStart()** method to provide code statements that you want InterSystems IRIS to execute on behalf of this host at production startup time. You can also implement the **OnProductionStop()** method.

For example, if your production requires different initial settings for property values, set the value in the **OnInit()** method of the business operation. For example, to change the initial setting of the `LineTerminator` property to depend on the operating system:

Class Member

```
Method OnInit() As %Status
{
    Set ..Adapter.LineTerminator="$Select($$isUNIX:$C(10),1:$C(13,10))"
    Quit $$$OK
}
```

13.10.3 Callbacks in Adapter Classes

An adapter class may override the **OnInit()** method. This method is called after the adapter object has been created and its configurable property values have been set. The **OnInit()** method provides a way for an adapter to perform any special setup actions.

For example, the following **OnInit()** method establishes a connection to a device when the adapter is started—assuming that this adapter also implements a **ConnectToDevice()** method:

Class Member

```
Method OnInit() As %Status
{
    // Establish a connection to the input device
    Set tSC = ..ConnectToDevice()
    Quit tSC
}
```

An adapter class can also override the **OnTearDown()** method. This method is called during shutdown before the adapter object is destroyed. The **OnTearDown()** method provides a way for an adapter to perform any special cleanup actions.

For example, the following **OnTearDown()** method closes a connection to a device when the adapter is stopped, assuming that this adapter also implements a method named **CloseDevice()**:

Class Member

```
Method OnTearDown() As %Status
{
    // close the input device
    Set tSC = ..CloseDevice()
    Quit tSC
}
```

13.11 Programmatically Working with Lookup Tables

InterSystems IRIS provides the [utility function](#) called `Lookup()` so that you can easily perform a table lookup from a business rule or DTL data transformation. This function works only after you have created at least one lookup table and have populated it with appropriate data.

For information on defining lookup tables, see [Defining Data Lookup Tables](#).

If you need more direct manipulation of lookup tables than the Management Portal provides, use the `Ens.Util.LookupTable` class. This class exposes lookup tables to access via objects or SQL. Additionally, it provides class methods to clear tables, export data as XML, and import data from XML.

`Ens.Util.LookupTable` provides the following string properties:

TableName

Name of the lookup table, up to 255 characters. You can view the lookup tables defined in a namespace by selecting **Interoperability**, **Configure**, and **Data Lookup Tables** in the InterSystems IRIS portal and then selecting **Open**.

KeyName

Key for the entry within the lookup table, up to 255 characters. This is the value from the **Key** field on the **Interoperability > Configure > Data Lookup Tables** page.

DataValue

Value associated with this key in the lookup table, up to 32000 characters. This is the value from the **Value** field on the **Interoperability > Configure > Data Lookup Tables** page.

A sample SQL query might be:

SQL

```
SELECT KeyName,DataValue FROM Ens_Util.LookupTable WHERE TableName = 'myTab'
```

`Ens.Util.LookupTable` also provides the following class methods:

%ClearTable()

Deletes the contents of the specified lookup table.

ObjectScript

```
do ##class(Ens.Util.LookupTable).%ClearTable("myTab")
```

%Import()

Imports lookup table data from the specified XML file. For the import to be successful, the file must use the same XML format as that provided by the **%Export()** method of this class.

ObjectScript

```
do ##class(Ens.Util.LookupTable).%Import("myFile.xml")
```

%Export()

Exports lookup table data to the specified XML file. If the file exists, InterSystems IRIS overwrites it with new data. If the file does not already exist, InterSystems IRIS creates it. The following example exports only the contents of the specified lookup table, myTab:

ObjectScript

```
do ##class(Ens.Util.LookupTable).%Export("myFile.xml", "myTab")
```

The following example exports the contents of all lookup tables in the namespace:

ObjectScript

```
do ##class(Ens.Util.LookupTable).%Export("myFile.xml")
```

The resulting XML file looks like the following example. Note that all entries, in all tables, appear as sibling <entry> elements inside a single <lookupTable> element.

XML

```
<?xml version="1.0"?>
<lookupTable>
  <entry table="myOtherTab" key="myKeyA">aaaaaa</entry>
  <entry table="myOtherTab" key="myKeyB">bbbbbbbbbb</entry>
  <entry table="myTab" key="myKey1">1111</entry>
  <entry table="myTab" key="myKey2">22222</entry>
  <entry table="myTab" key="myKey3">333333</entry>
</lookupTable>
```

For each <entry>, the *table* attribute identifies the table that contains the entry. The *key* attribute gives the name of the key. The text contents of the <entry> element provide the entry's value.

In addition to the XML format described above, you can use the SQL Import Wizard to import comma-separated value (CSV) files that list tables and keys.

13.12 Defining a Custom Archive Manager

For InterSystems IRIS, the Management Portal provides a tool called the Archive Manager; this is described in [Using the Archive Manager](#). You can define and use a custom Archive Manager. To do so, create a class as follows:

- It can use Ens.Archive.Manager as a superclass.
- It must define the DoArchive() method, which has the following signature:

```
ClassMethod DoArchive() As %Status
```

An alternative option is to use the Enterprise Message Bank, which enables you to archive messages from *multiple* productions. For an overview, see [Defining the Enterprise Message Bank](#).

14

Testing and Debugging Productions

This topic explains the facilities available for testing and debugging productions. The information is also useful for troubleshooting and tuning production software that is already in use at the enterprise.

14.1 Correcting Production Problem States

If a production is [Suspended](#) or [Troubled](#), read this section.

14.1.1 Suspended Productions

A suspended production occurs when a production is stopped before all asynchronous messages in the queue can be processed. If you do not manually clear these asynchronous messages, they are automatically processed when the production is started back up. If you want the messages to be processed, no other steps are required before starting a suspended production.

14.1.2 Recovering a Troubled Production

A production acquires a status of Troubled if InterSystems IRIS is stopped but the production did not shut down properly. This can happen if you restarted InterSystems IRIS® data platform or rebooted the machine without first stopping the production.

In this case the **Recover** command appears on the **Production Configuration** page. Click **Recover** to shut down and clean up the troubled instance of the production so that you can run a new instance when you are ready.

Or you may need to use the command line to recover the production. See [Using Ens.Director to Start and Stop a Production](#).

14.1.3 Resetting Productions in a Namespace

During development, you might want to be absolutely sure that all queues for a production have been cleared or to remove all information about a production before starting another one. The **CleanProduction()** method clears the queues.

CAUTION: Never use this procedure on a live, deployed production. The **CleanProduction()** method removes all messages from queues and removes all current information about the production. Use this procedure only on a production that is still under development.

To use the **CleanProduction()** method:

1. Change to the appropriate namespace:

ObjectScript

```
set $namespace = "EnsSpace"
```

Where EnsSpace is the name of the production-enabled namespace where the production runs.

2. Enter the following command:

ObjectScript

```
do ##class(Ens.Director).CleanProduction()
```

14.2 Testing from the Management Portal

You can use the Management Portal to perform several tasks as you develop, test, and debug your productions:

- Ability to view and modify system configuration.
- Ability to start and stop a production.
- Ability to view queues and their contents; messages and their details; adapters and actors and their status; business processes and their status; code and graphical representations of configured items.
- Ability to view, sort, and selectively purge Event Log entries.
- Ability to suspend (and later resend) messages whose connectivity is temporarily blocked.
- Ability to filter and search the message warehouse for specific messages, by category or message content, using a graphical user interface or by entering SQL SELECT commands.
- Ability to visually trace message activity using a graphical user interface.
- Ability to create and view statistical reports.

Portal features that are most useful for developers are the Monitor Service, which constantly collects runtime data, the Testing Service, which you can use to issue simulated requests into a production that you are developing, and the Event Log, which logs the status messages issued by business hosts. Use these features together to generate test data and study the results.

For information on using the portal, see [Managing Productions](#).

The Management Portal has a Test menu where you can test both business hosts and data transformations. It contains the following items:

- Business Hosts—The **Interoperability > Test > Business Hosts** page allows you to test business processes and business operations.
- Data Transformations—This option brings you to another page, where you can select a data transformation and click Test. For details, see the Testing Data Transformations section.

14.2.1 Using the Testing Service

The Testing Service allows you to test a business process or business operation of a running production in the active namespace.

Before testing a business process or business operation:

- Make sure the appropriate production is running. See [Starting and Stopping Productions](#).

- Make sure that testing is enabled for this production. From the **Production Configuration** page:
 1. Select the **Production Settings** link.
 2. On the **Settings** tab, open the **Development and Debugging** property list and check the **Testing Enabled** check box.
 3. Select **Apply**.

You can navigate to the Testing Service from the following locations in the Management Portal:

- Select **Interoperability > Test** and then select either **Business Hosts** or **Data Transformation**.
- From the configuration diagram on the **Production Configuration** page, select a business process or business operation in the left pane and select **Test** on the **Actions** tab.

To use the Testing Service on a Business Process or Business Operation:

1. In the Management Portal, select **Interoperability > Test > Business Hosts** to display the **Testing Service** page.
This page provides options that let you select either a **Business Process** or **Business Operation** as the target of your testing.
2. Select either **Business Process** or **Business Operation** as appropriate.
3. Select the testing target from the drop-down list.
4. Select the type of message to send. The page displays the following fields:
 - **Current Production**—The name of the currently running production (view-only).
 - **Target**—The business process or business operation that you selected in the previous Testing Service page (view-only).
 - **Request Type**—Select from a list of request messages. Only the request types that are valid for the **Target** are listed, including subclasses of supported types.

The system now displays a form you can use to specify values for the properties of the message type you selected.

5. Enter values for the properties of the message. If the request message type has no properties, none are displayed.
By default, this table cannot display more than 99 properties. To increase the number of properties that it can display, enter the following command in the ObjectScript shell in the namespace where this maximum should be increased:

ObjectScript

```
set ^CSP.AutoFormMaxProperties=newmaximum
```

Where *newmaximum* is an integer that specifies the maximum number of properties to display. (This change also affects the **Body** tab of the Message Viewer and the Visual Trace; see [Controlling the Display](#).)

If you are testing a virtual document message, there is a free-form box where you can paste test message content. Below this box you can enter object properties for the message.

6. Select **Invoke** to submit the request with the values you entered and view the results.
If time elapses while the Testing Service attempts the request, a **Waiting** page displays the following view-only information:
 - **Target**—The session ID associated with the request.
 - **Request Type**—The request type of the selected target.
 - **Session Id**—The session ID associated with the request.
 - **Request Sent**—The date and time when the request was sent.

- **Response received**—The status *Waiting* and a graphical progress bar, indicating that work is being done.

Finally, the *Results* page displays any output values from the response generated by your request, including any errors with the full error message text.

You can perform one of the following commands when the test completes:

- Select **Done** to return to the home page.
- Select **Trace** to navigate to the Visual Trace page to visually follow the path of the message through the production.

You can also use the classes and methods in the `EnsLib.Testing` package. See `EnsLib.Testing.Service` for details.

14.3 Debugging Production Code

The first step in debugging is to enable tracing as described in [Enabling Tracing](#). If this does not reveal the problem, you can step into the code using the debugger, as follows:

1. Edit the code in an IDE to insert the *BREAK* command where you want to start debugging.
2. Enable the **Foreground** setting for the business host that uses the class you want to debug.
3. Start the production. The job that you marked in Step 2 runs in the foreground in the Terminal.
4. When that *BREAK* command is reached, the Terminal enters debug mode and you can step through the code.

For details, see [Command-Line Routine Debugging](#).

14.4 Enabling %ETN Logging

The Event Log automatically includes partial information about system-level exceptions (including exceptions in your code). These Event Log entries end as follows by default:

```
-- logged as '-' number - '@' quit arg1/arg2 }'
```

To get more complete information about such errors:

1. Set the `^Ens.Debug("LogETN")` global node to any value.

This causes InterSystems IRIS to record additional details for system-level exceptions.

2. Rerun the code that you think caused the exception (for example, resend messages).
3. Recheck the Event Log, which now contains entries that end as follows:

```
-- logged as '25 Sep 2012' number 15 '@' quit arg1/arg2 }'
```

This information refers to an entry in the Application Error Log—specifically it refers to error 15 in the Application Error Log for 25 September 2012.

4. Then to examine these exceptions, you can either:
 - Select **System Operation > System Logs > Application Error Log**.
 - Use the `^%ERN` routine. For details, see [Other Debugging Tools](#).

15

Deploying a Production

Typically, you develop a production on a development system and then, after completing and testing the production on a test deployment, you deploy it on a live production system. This page describes how to use the Management Portal to package a deployment from a development system and then to deploy it on another system. It also describes how you can develop and test changes to a production and then deploy those updates to a system running with live business data.

15.1 Overview of Deploying a Production

You can deploy a production using either the Management Portal or an IDE. The Management Portal automates some steps that you need to perform manually using an IDE. If you have a live production that is being used and are developing updates to the production, you need to ensure that the live production is updated without interrupting your processing of business data. At its simplest level deploying a production is done by exporting the XML definition of the production from one system and importing and compiling the XML on a target system. The most important issues for a successful deployment from development to live systems are:

- Ensuring that the XML deployment file has all required components.
- Testing the deployment file on a test system before deploying it to the live system.
- Ensuring that the deployment file is loaded on the target system without disrupting the live production.

Typically, deploying a production to a live system is an iterative process, with the following steps:

1. Export the production from the development system.
2. Deploy the deployment file on a test system.
3. Ensure that the production has all required components and runs properly on the test system. If any failures are found fix them and repeat step 1.
4. After the production has been deployed to the test system without errors, deploy the deployment file to the live system. Monitor the live system to ensure that the production continues to run correctly.

You should ensure that the test system environment matches as closely as possible the environment of the live system. If you are updating an existing production, the production on the test system should match the production on the live system before the update is applied. If you are deploying a production on a new InterSystems IRIS® installation, the test system should be a new InterSystems IRIS installation.

In order to update a component in a running production, you must do the following:

1. Load the updated XML on the system.

2. Compile the XML.
3. Update the running instances of the component to the new code by disabling and re-enabling the component.

The deployment process is slightly different depending on whether or not the target system is already running a version of the production. If the target system is running an older version of the production, then the deployment file should contain only the updated components and some configuration items, and, in most cases, it should not contain the definition of the production class. If the target system does not contain the production, the deployment file should contain all production components and settings. If you use the **Interoperability > Manage > Deploy Changes > Deploy** Management Portal page to deploy updates to a running production, the portal automatically does the following:

1. Creates a rollback and log file.
2. Disables components that have configuration items in the deployment file.
3. Imports and compiles the XML. If there is a compilation error, the portal automatically rolls back the deployment.
4. Enables the disabled components

There are some conditions where you have to explicitly stop and restart a component or the entire production. If you are using an IDE or importing the classes from the Management Portal **System Explorer**, then you have to perform these steps manually.

In order to export and deploy a production, you must have the appropriate permissions, for example:

- `%Ens_Deploy:USE` to access to the **Interoperability > Manage > Deployment Changes** page and deployment actions
- `%Ens_DeploymentPkg:USE` to export the XML to the server
- `%Ens_DeploymentPkgClient:WRITE` to export the XML locally using the web browser
- `%Ens_DeploymentPkgClient:USE` to deploy the XML using the web browser

By default, these resources are granted automatically only to users with the role `%EnsRole_Administrator`. For more information, see [Ensemble Resources to Protect Activities](#).

15.2 Exporting a Production

To export the XML for a production using the Management Portal, open the production, click **Production Settings** and the **Actions** tab and then click the **Export** button. InterSystems IRIS selects all business services, business processes, business operations, and some related classes, and then displays a form to allow you to add export notes and additional components. This form has the following areas:

- The upper left provides read-only information on the export, including the following details:
 - **Production** — Full name of the production class, including the package or packages.
 - **Namespace** — Namespace in which the export is being performed.
 - **Instance** — Name of the instance in which the export is being performed.
 - **Machine** — Name of the machine on which the export is being performed.
 - **User** — Name of the user performing the export.
- In the upper right, the **Export Notes** field lets you record any notes to include within the export.
- The middle area lets you choose additional items to include in the export package:

Add to package:

Config. Item Definition	Business Service Class	Business Process Class	Business Operation Class	Message Class	
Rule Definition Class	Data Transformation Class	VDoc Schema Category	Lookup Table	Dashboards	Class
Studio Project Files	Production Settings	Deployable Settings			

When you click any button in this area, InterSystems IRIS then displays a dialog box where you can choose items of that type to include.

- The lower area lists the items that you have selected to export. These items are shown in two groups: **Manually Added** (which shows the items you manually added as described in the previous bullet), and **Production** (which shows the default items to export).

You can also export a business service, process, or operation by selecting the component in the production configuration and then clicking the **Export** button on the **Actions** tab. In both cases, you can add additional components to the package by clicking on one of the buttons and selecting a component. You can remove components from the package by clearing the check box.

You can use the export notes to describe what is in the deployment package. For example, you can describe whether a complete production is in the package or set of components that are an update to a production. The export notes are displayed when you are deploying the package to a target system using the Management Portal.

When you are exporting a deployment package, the first decision you should make is whether the target system has an older version of the production.

If you are deploying the production as a new installation, you should:

- Include the definition of the production class.
- Include the production settings.
- Include the definitions of all components used in the production.
- Exclude the production settings (ptd file) for each component. This would duplicate the definition in the production class.

If you are deploying the production to update a live version of the production, you should:

- Exclude the definition of the production class.
- Exclude the production settings unless there are changes and you want to override any local settings.
- Include the definition of all components that have been updated.
- Include the production settings (ptd) file for any component whose setting have been changed or that should be disabled before the XML is imported and compiled.

Although many components are included by default in the package, you have to add others manually by selecting one of the buttons in the **Add to package** section. For example, if any of the following are used in your production, you need to add them manually:

- Record maps—the defined and generated classes are included.
- Complex record maps—the defined and generated classes are included.
- Lookup tables
- User classes referenced in code
- System default settings or schedule specifications that are set as deployable

The **Production Settings** button allows you to add the production ptd file. This XML defines the following:

- Production comments
- General pool size
- Whether testing is enabled and whether trace events should be logged.

You can deselect any component in the list by clearing its check box. You can select a component by checking its box. The **Select All** button checks all the boxes and the **Unselect All** button clears all check boxes.

Once you have selected the components for the deployment package, create it by clicking **Export**. You can save the export file to the server or locally via the browser's downloading capability. If you export it to the server, you can specify the file location. If you export it via the web browser, you can specify the file name.

The deployment package contains the following information about how it was created:

- Name of the system running InterSystems IRIS
- Namespace containing the production
- Name of the source production
- User who exported the production
- UTC timestamp when the production was exported

You should keep a copy of the deployment file on your development system. You can use it to create a new deployment package with the latest changes to the components. Keeping a copy of the deployment file saves you from having to manually select the components to be included in the deployment file.

To create a new deployment package using an existing deployment package to select the components, do the following:

1. On the development system with the updated production, click **Production Settings** and the **Actions** tab and then the **Re-Export** button.
2. Select the file containing the older deployment package.
3. InterSystems IRIS selects the same components from the current production that were included in the older deployment package.
4. If there were any components missing from the older deployment package or if you have added new components to the production, add the missing components manually.
5. Click the **Export** button to save a new deployment package with the updated components.

Note: If a production uses XSD schemas for XML documents or uses an old format schema for X12 documents, the schemas are not included in the XML deployment file and have to be deployed through another mechanism. InterSystems IRIS can store X12 schemas in the current format, in an old format, or in both formats. When you create a deployment file, it can contain X12 schemas in the current format, but it does not contain any X12 schemas in the old format or any XSD schemas for XML documents. If your production uses an old format X12 schema or uses any XSD XML schema, you must deploy the schemas independently of deploying the production. For the schemas that are not included in the deployment file, they can be deployed to a target system by either of the following means:

- If the XML or X12 schema was originally imported from an XSD or SEF file and that file is still available, import the schema on the target system by importing that file. XSD files can be used to import XML schemas and SEF files can be used to import X12 schemas.
- Export the underlying InterSystems IRIS global that contains the schema and then import this on the target system. To export a global, select **System Explorer > Globals**, select the desired globals and then select **Export**. The X12 schemas are stored in the `EnsEDI.Description`, `EnsEDI.Schema`, `EnsEDI.X12.Description`, and `EnsEDI.X12.Schema` globals. The XML schemas are stored in the `EnsEDI.XML.Schema` global. See [Exporting Globals](#) for details on exporting globals.

15.3 Deploying a Production on a Target System

The Management Portal automates the process of deploying a production from a development system to a live system. This section describes what InterSystems IRIS does when you are loading a new version of a production on a live system.

Once you have the deployment package XML file, you can load it on a target system. In the Management Portal, select the correct namespace and click **Interoperability, Manage, Deployment Changes, Deploy**, and then click the **Open Deployment** or **Open Local Deployment** button, depending on whether the XML deployment package is located on the server or on the local machine. The **Open Local Deployment** button is not active if you are on the server machine. After you select the XML deployment package file, the form lists the new and changed items in the deployment package, displays the deployment notes that were specified when the package was created and allows you to specify the following deployment settings:

- **Target production**—specifies the production that the components will be added to. If the deployment package includes the production class from the source production, then the target production is set to the source production and cannot be changed. Otherwise, InterSystems IRIS sets the default production to the currently open production, but allows you to change it.
- **Rollback file**—specifies the file to contain the rollback information. The rollback file contains the current definitions of all components that are being replaced by the deployment.
- **Deployment log file**—contains a log of the changes caused by the deployment.

When you have read the deployment notes and made any changes to the deployment settings, complete the deployment by clicking the **Deploy** button. As part of deploying the package, InterSystems IRIS does the following to stop the production, load the new code, and then restart the production.

1. Create and save the rollback package.
2. Disable the components in the production that have a production settings (ptd) file in the deployment package.
3. Import the XML file and compile the code. If there is an error compiling any component, the entire deployment is rolled back.
4. Update the production settings.
5. Write a log detailing the deployment.

6. Enable the production components that were disabled if their current setting specify that they are enabled.

To undo the results of this deployment change, use the **Open Deployment** button to select the rollback file, then click the **Deploy** button.

A

Enabling Tracing Code

This topic describes how to enable tracing code, view trace messages, and log trace messages.

A.1 Overview

Tracing is a tool for use primarily during development. Trace elements enable you to see the behavior of various elements in a production, for the purpose of debugging or diagnosis. You typically disable tracing before a production goes live.

The production trace mechanism works as follows:

- As part of the development process, InterSystems IRIS® developers add trace elements to the appropriate areas of your code. These trace elements (potentially) write trace messages at runtime. See [Adding Trace Elements](#).

Note that these are messages only in a general sense; trace messages are simply strings and are unrelated to `Ens.Message` and its subclasses.
- As part of the configuration process, do the following:
 - Configure the production to [enable tracing](#). This step means that, at runtime, the trace elements are executed (rather than being ignored).
 - Optionally [enable logging](#) for the trace messages. This step writes trace messages to the [Event Log](#).
 - Optionally configure the applicable business hosts to run in the foreground so that you can see trace messages in the Terminal while the production is running. See the [last section](#).

You typically disable tracing before a production goes live.

A.2 Enabling Tracing

By default, all user trace elements are enabled. You can also enable tracing of various system events.

To do so, set values for some or all of the following nodes of the `^Ens . Debug` global:

Node	Purpose
<code>^Ens.Debug("TraceCat")</code>	Controls tracing overall as follows: - If this node is not set, only user trace elements are enabled. In this case, you can enable specific kinds of system tracing by setting subnodes, as described in the rest of this table. - If the value of this node is 0, no tracing is enabled. - If the value of this node is 1, all tracing is enabled, apart from any explicitly disabled kinds of tracing.
<code>^Ens.Debug("TraceCat", "bproc")</code>	Enables or disables system traces from business processes. For this node and all the rest of the nodes in this table, if the node value is 1, the specified traces are enabled. If the node value is 0, these traces are disabled. This node is ignored if the parent node value is 0.
<code>^Ens.Debug("TraceCat", "connwait")</code>	Enables or disables system traces from adapters waiting to connect.
<code>^Ens.Debug("TraceCat", "exterr")</code>	Enables or disables system traces showing errors from external systems.
<code>^Ens.Debug("TraceCat", "file")</code>	Enables or disables system traces from file read or write operations.
<code>^Ens.Debug("TraceCat", "ontask")</code>	Enables or disables system traces from business host framework events.
<code>^Ens.Debug("TraceCat", "parse")</code>	Enables or disables system traces from virtual document parsers.
<code>^Ens.Debug("TraceCat", "protocol")</code>	Enables or disables system traces of sequence numbers from the MSH segment in HL7 messages.
<code>^Ens.Debug("TraceCat", "queue")</code>	Enables or disables system traces related to message queue management.
<code>^Ens.Debug("TraceCat", "sql")</code>	Enables or disables system traces from processing events of the SQL adapter.
<code>^Ens.Debug("TraceCat", "system")</code>	Enables or disables general system trace elements.
<code>^Ens.Debug("TraceCat", "timing")</code>	Enables or disables system traces providing information about duration of calls.
<code>^Ens.Debug("TraceCat", "transform")</code>	Enables or disables system traces about DTL data transformations, apart from errors.
<code>^Ens.Debug("TraceCat", "user")</code>	Enables or disables user traces.
<code>^Ens.Debug("TraceCat", "xform")</code>	Enables or disables system traces about errors in DTL data transformations.

For example, to enable tracing related to message queue management, enter the following command in the Terminal, in the appropriate namespace:

```
set ^Ens.Debug("TraceCat","queue")=1
```

Also see [Enabling %ETN Logging](#).

A.3 Enabling Logging for Trace Messages

InterSystems IRIS can also log trace messages (that is, write them to the [Event Log](#)). To enable or disable logging of trace messages, use the following settings:

- For any business host, use the [Log Trace Events](#) setting. When this setting is selected, InterSystems IRIS logs all the enabled trace messages for this business host log.
- For the production, use the [Log General Trace Events](#) setting. When this setting is selected, InterSystems IRIS logs all enabled trace messages from production elements that are *not* business hosts.

There is no overlap or interaction between these settings; [Log General Trace Events](#) does not override or provide a default value for [Log Trace Events](#).

See [Settings in All Productions](#).

A.4 Seeing Trace Messages in the Terminal

To see the trace messages in the Terminal, enable the [Foreground](#) setting for the business host or business hosts in which you are interested.

When you run the production, InterSystems IRIS opens a Terminal window for each foreground business host. This Terminal window shows all enabled trace messages for that business host. It also shows all log items and alerts.

B

Life Cycle of a Production and Its Parts

This page describes the life cycle of a production and its component parts, for reference.

B.1 Life Cycle of a Production

B.1.1 Production Startup

When a production starts, the sequence of actions is as follows:

1. The production class is instantiated; its optional **OnStart()** method executes.
2. The production instantiates each business operation and executes its optional **OnProductionStart()** method.
3. The production instantiates each business process and executes its optional **OnProductionStart()** method.
4. The production clears the business metric cache of any metric values left over from a previous run.
5. The production instantiates each business service and executes its optional **OnProductionStart()** method.
6. The production processes any items already placed in queues. This includes asynchronous messages that were queued when the production stopped.
7. The production now accepts input from outside InterSystems IRIS® data platform.

B.1.2 Production Shutdown

When a production stops, the sequence of actions is as follows:

1. The production takes each business service offline and executes its optional **OnProductionStop()** method. This action stops all requests from outside InterSystems IRIS.
2. All business hosts receive a signal to become Quiescent.
3. All queues go into a Quiescent state. This means that from this point forward, business hosts can only process queued messages with High priority (synchronous messages). Asynchronous messages remain on their respective queues.
4. The production finishes processing all synchronous messages to the best of its ability.
5. The production takes each business process offline and executes its optional **OnProductionStop()** method.
6. The production takes each business operation offline and executes its optional **OnProductionStop()** method.

7. The production goes offline. InterSystems IRIS executes the optional **OnStop()** method in the production class.

B.2 Life Cycle of a Business Service and Adapter

B.2.1 Production Startup

When you start a production (or change the configuration of a specific business service), InterSystems IRIS automatically performs the following tasks for each configured business service class (that is, for every business service listed in the production definition):

1. InterSystems IRIS invokes the business service's **OnProductionStart()** callback method, if defined.

The **OnProductionStart()** method is a class method that is invoked once for each business service class listed in the production configuration. A business service class can use this callback to perform any class-wide initialization it may require. If the business service does not have an adapter, the business service class can use this callback to check for errors.

2. InterSystems IRIS creates one or more background processes in which to execute the business service.

The number of background processes is determined by the business service's `PoolSize` property within the production configuration. Each background process is referred to as an instance of the business service and contains an instance of a business service object.

InterSystems IRIS only creates a background process for a business service if the following conditions are true:

- The business service class must have an associated inbound adapter class as specified by its `ADAPTER` class parameter.
A business service class with no associated inbound adapter is referred to as an “adapterless service.” Instead of waiting for external events, such a service is invoked in-process (perhaps by a composite application).
- The business service's `Enabled` property within the production configuration must be set to 1 (otherwise the business service is considered to be disabled and will not accept input).
- The business service's `PoolSize` property within the production configuration must be set to a value greater than 0.

If the business service's `Foreground` property within the production configuration is set to 1, then InterSystems IRIS will create a foreground process (that is, InterSystems IRIS will create a Terminal window) for the business service. This feature facilitates testing and debugging.

3. InterSystems IRIS initializes the system monitoring information used to monitor the status and operating history of the business service.
4. Within each background process, InterSystems IRIS does the following:
 - a. Creates an instance of the business service class.
 - b. Supplies the most recently configured values of any of the business service's settings
 - c. Invokes the business service's **OnInit()** callback method (if present). The **OnInit()** method is an instance method that provides a convenient place to execute any initialization logic for a business service.
 - d. Creates an instance of the associated adapter class (if one is defined) and supplies the most recently configured values of any of the adapter's settings.

B.2.2 Runtime

While the production is running, the business service repeatedly calls the inbound adapter's **OnTask()** method. This OnTask loop is controlled by the business service's CallInterval setting and %WaitForNextCallInterval property as follows:

1. The business service calls the inbound adapter's **OnTask()** method.
2. **OnTask()** checks outside the InterSystems IRIS production for input events of interest to the business service:
 - If it finds input, **OnTask()** calls the internal **ProcessInput()** method of the associated business service object.
 - If it does not find input, **OnTask()** returns control to the business service, which waits for the next CallInterval to elapse before returning to step 1.
 - Multiple input events may exist. For example, if the business service uses File.InboundAdapter, there may be several files waiting in the designated directory.

If there are multiple input events:

- Typically, the **OnTask()** method calls the **ProcessInput()** method as many times as is necessary to process all the available input events until no more are found.
- Alternatively, an inbound adapter can restrict **OnTask()** to call **ProcessInput()** only once per CallInterval, even if multiple input events exist. Rather than processing all the input events, **OnTask()** goes to sleep after processing the first event found.

3. The internal **ProcessInput()** method sets the business service %WaitForNextCallInterval property to 0 (false) and calls **OnProcessInput()** to handle the input event.
4. Upon completion, the internal **ProcessInput()** method returns control to **OnTask()**.
5. At this point, **OnTask()** may set %WaitForNextCallInterval to 1 (true). This restricts the business service to processing only one input event per CallInterval, even when multiple input events exist.

Usually you want the business service to process all available input events without delay, so usually you do not want to do anything to change %WaitForNextCallInterval at this step. It should retain the 0 (false) value set by the internal **ProcessInput()** method.

The adapter base class Ens.InboundAdapter has an **OnTask()** method that calls the internal **ProcessInput()** method, sets %WaitForNextCallInterval to 1, and returns.

Tip: If you simply want a business service to wake up and run its **ProcessInput()** method once per CallInterval without concern for events outside InterSystems IRIS, use the adapter class Ens.InboundAdapter.

6. **OnTask()** returns.
7. The business service tests the value of its %WaitForNextCallInterval property:
 - If 1 (true), the business service waits for the CallInterval to elapse before returning to step 1.
 - If 0 (false), the business service immediately returns to step 1. The CallInterval does not come into play until **OnTask()** discovers there is no more input (see step 2).

B.2.3 Production Shutdown

When a production stops, the following events related to business services occur:

1. InterSystems IRIS disables each business service; no more incoming requests are accepted for this production.

2. The **OnTearDown()** method in each inbound adapter is called.
3. All inbound adapter and business service objects are destroyed and their background processes are killed.
4. Each business service's **OnProductionStop()** class method is called, once for each configured item of that class in the production.

When a business service is disabled by a system administrator, or becomes inactive according to its configured schedule, the production continues to run but the associated inbound adapter is shut down, and its **OnTearDown()** method is executed.

B.3 Life Cycle of a Business Process

Each time a production starts, InterSystems IRIS creates the public actor pool for the production. The value of the ActorPoolSize setting determines the number of jobs in the pool.

Within each job in the actor pool, there is an instance of an Ens.Actor object whose responsibility it is to manage the use of its job by business processes. This Ens.Actor instance is called an *actor*.

The business processes in a production can share the public message queue called Ens.Actor. This public queue is the target of all messages sent to any business process within a production that does not have its own, private queue. Actors listen on the Ens.Actor queue whenever they are free to host a business process. When a request arrives on the Ens.Actor queue, any actor that is free may assign its job to host the business process named in the request. Requests on the Ens.Actor queue are processed in the order in which they are received. Each successive request is claimed by the next available actor, on an ongoing basis.

For information on pools, see [Pool Size and Actor Pool Size](#).

Unlike the lifecycle of business services and business operations, the OnInit and OnTearDown methods of a business process are not called when the business process is started or stopped. Rather, these methods are executed every time a request is processed by the business process (InProc or Queued). You can implement the OnProductionStart method of the business process class to execute custom code when the production is started or when the business process is restarted. The class' OnProductionStop method is called when the production is stopped.

B.3.1 Life Cycle in the Public Queue

The life cycle of a business process that uses the public queue is as follows:

1. The initial request is addressed to the business process. The request message arrives on the Ens.Actor queue.
2. As soon as an actor becomes available, it pulls the request message off the Ens.Actor queue.
3. The actor creates a new instance of the appropriate business process class. The **OnInit()** method of the business process is invoked. The actor supplies the most recent values of the settings of the business process and invokes the **OnRequest()** method of that instance. The business process instance is now ongoing.
4. While the business process instance is ongoing, whenever it is not active (for example, while it is waiting for input or feedback), it generally returns control to the actor that instantiated it. When this happens:
 - a. The actor suspends the business process instance.
 - b. The actor saves the current state of the instance to disk.
 - c. The actor returns its job to the actor pool.
5. Whenever an ongoing business process has no assigned actor and a subsequent request or response arrives addressed to the business process (for example, when the anticipated input or feedback arrives) the following sequence occurs:

- a. When an actor becomes available, it pulls the new request or response message from the `Ens.Actor` queue.
 - b. The actor restores the corresponding business process object from disk, complete with all of its state information.
 - c. The actor invokes the **OnRequest()** or **OnResponse()** instance method, as appropriate.
6. After the business process instance completes execution, its current actor invokes its **OnComplete()** method and marks the instance with the status of *IsComplete*. The actor also returns its job to the actor pool. No further events are sent to this business process instance. The business process' **OnTearDown** method is invoked.

B.3.2 Life Cycle in a Private Queue

Alternatively, you can configure business processes to have private queues, bypassing the public `Ens.Actor` queue. The life cycle of a business process with a private queue runs exactly as described for a public queue, except that:

- The business process runs in jobs from the private pool only.
- The messages addressed to this business process arrive on its own, private queue, and do not arrive on the `Ens.Actor` queue.

B.4 Life Cycle of a Business Operation and Adapter

InterSystems IRIS automatically manages the life cycle of each business operation.

B.4.1 Production Startup

When you start a production (or change the configuration of a specific business operation), InterSystems IRIS automatically performs the following tasks for each configured business operation class (that is, for every business operation listed in the production definition):

1. It invokes the class' **OnProductionStart()** callback method, if defined.

The **OnProductionStart()** method is a class method that is invoked once for each business operation class listed in the production configuration. A business operation class can use this callback to perform any class-wide initialization it may require.

2. It creates one or more background processes in which to execute the business operation.

The number of background processes is determined by the business operation's `PoolSize` property within the production configuration. Each background process is referred to as an instance of the business operation and contains an instance of a business operation object.

InterSystems IRIS will only create a background process for a business operation if the following conditions are true:

- The business operation class must set its *INVOCATION* class parameter to `Queue`.
- The business operation's `Enabled` property within the production configuration must be set to 1 (otherwise the business operation is considered to be disabled). A disabled business operation still has an incoming message queue. Any requests posted to this queue will not be processed until the business operation is enabled.
- The business operation's `PoolSize` property within the production configuration must be set to a value greater than 0.

If the business operation's Foreground property within the production configuration is set to 1, then InterSystems IRIS will create a foreground process (that is, it will create a Terminal window) for the business operation. This feature facilitates testing and debugging.

3. It initializes the system monitoring information used to monitor the status and operating history of the business operation.
4. Within each background process:
 - a. InterSystems IRIS creates an instance of the business operation class, supplies the most recently configured values of any of the business operation's settings, and invokes the business operation **OnInit()** callback method (if present). The **OnInit()** method is an instance method that provides a convenient place to execute any initialization logic for a business operation.
 - b. InterSystems IRIS creates an instance of the associated adapter class (if one is defined) and supplies the most recently configured values of any of the adapter's settings.

B.4.2 Runtime

At runtime, a business operation does the following:

1. It waits for requests to be sent (from business services, business processes, and other business operations) to its associated message queue.
2. After the business operation retrieves a request from its message queue, it searches its message map for the operation method that corresponds to the request type. It then invokes that operation method.
3. The operation method, using the data within the request object, makes a request to an external application. Typically it does this by calling methods of its associated outbound adapter object.

B.4.3 Production Shutdown

When a production stops, the following events related to business operations occur:

1. InterSystems IRIS waits for each business operation to reach a quiescent state (that is, InterSystems IRIS waits until each business operation has completed all of its synchronous requests).
2. The **OnTearDown()** method in each outbound adapter is called.
3. All outbound adapter and business operation objects are destroyed and their background processes are killed.
4. Each business operation's **OnProductionStop()** class method is called, once for each configured item of that class in the production.

When a business operation is disabled by a system administrator, or becomes inactive according to its configured schedule, the production continues to run but the associated outbound adapter is shut down, and its **OnTearDown()** method is executed.